

Kubernetes Practice

Dept. of smart finance
Korea Polytechnics

KUBERNETES 개요

DEFINITION

컨테이너화된 애플리케이션의 배포·확장·운영을 자동화하는 오픈소스
컨테이너 오케스트레이션 플랫폼

NECESSITY

컨테이너 수 급증에 따른 배포 관리, 장애 복구, 자동 확장 및 네트워크
운영 자동화 필요성 대두

CORE FUNCTIONS

자동 배포 & Self-Healing

Scaling & Load Balancing

Service Discovery

Rolling Update

CLUSTER STRUCTURE

Control Plane

API Server, Scheduler, etcd

Worker Node

Pod (Container) 실행 환경

HIERARCHY

Cluster

>

Node

>

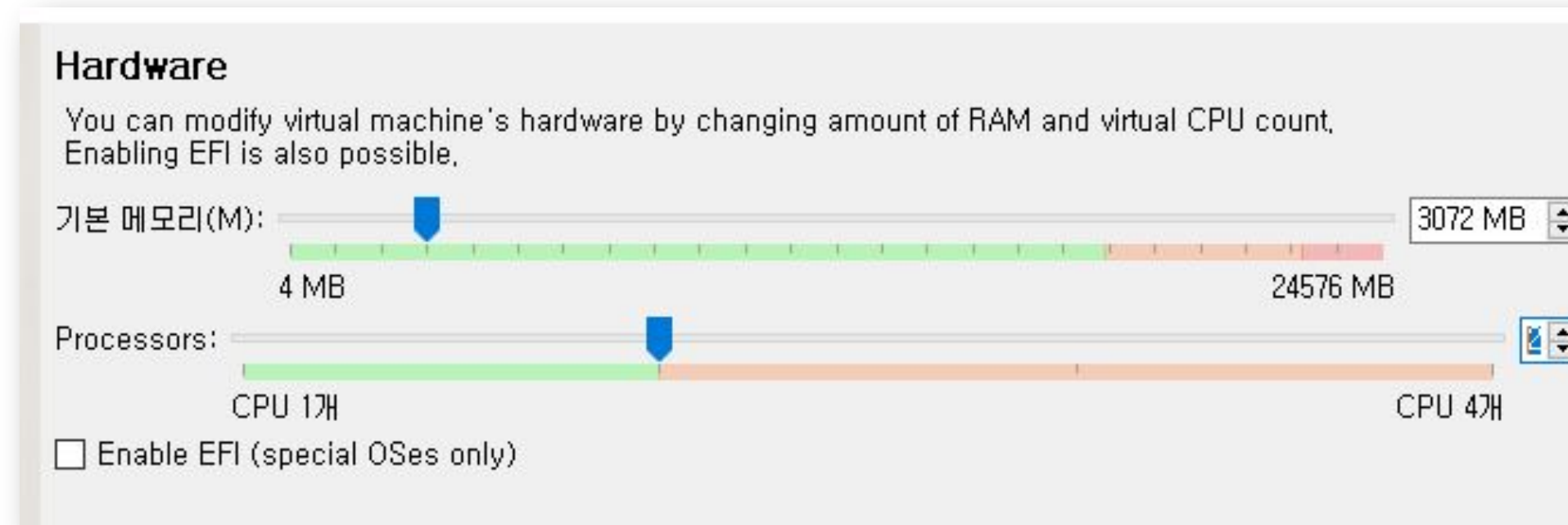
Pod

CORE SUMMARY

여러 서버에 분산된 컨테이너를 하나의
클러스터로 묶어 자동으로 배포·확장·복구·
관리하는 핵심 플랫폼입니다.

Requirements

- We'll make 2 VMs, one is master node and the other is worker node.
Each node need to be set as below
- CPU : 2 core (minimum)
- RAM : 3GB (minimum)
- Storage: 30GB(minimum)
- OS : Ubuntu 24.04 (preferred)



Network setup

In virtual box

- 1. Set a host network manager (menu -> file -> host network manager...)

이름	IPv4 Prefix	IPv6 Prefix	DHCP 서버
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24		사용 안 함

어댑터(A) DHCP 서버(D)

☐ 자동으로 어댑터 설정(A)

☒ 수동으로 어댑터 설정(M)

IPv4 주소(I): 192.168.56.1

IPv4 서브넷 마스크(M): 255.255.255.0

IPv6 주소(P): fe80::ec4f:71bf:e3b1:cf05

IPv6 접두사 길이(L): 64

적용 Reset

어댑터(A) DHCP 서버(D)

☐ Enable Server

서버 주소(R): 192.168.56.100

서버 마스크(M): 255.255.255.0

최저 주소 한계(L): 192.168.56.101

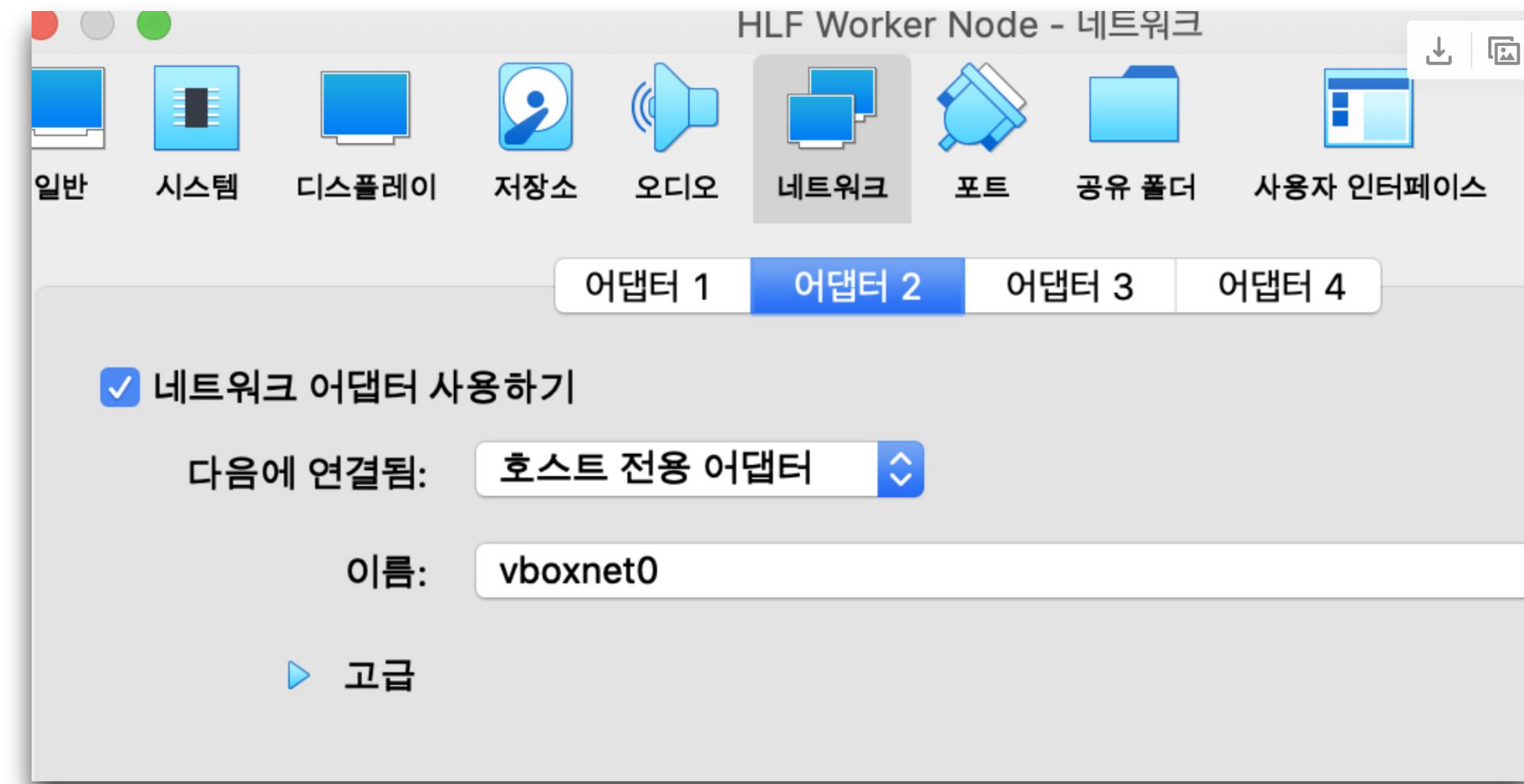
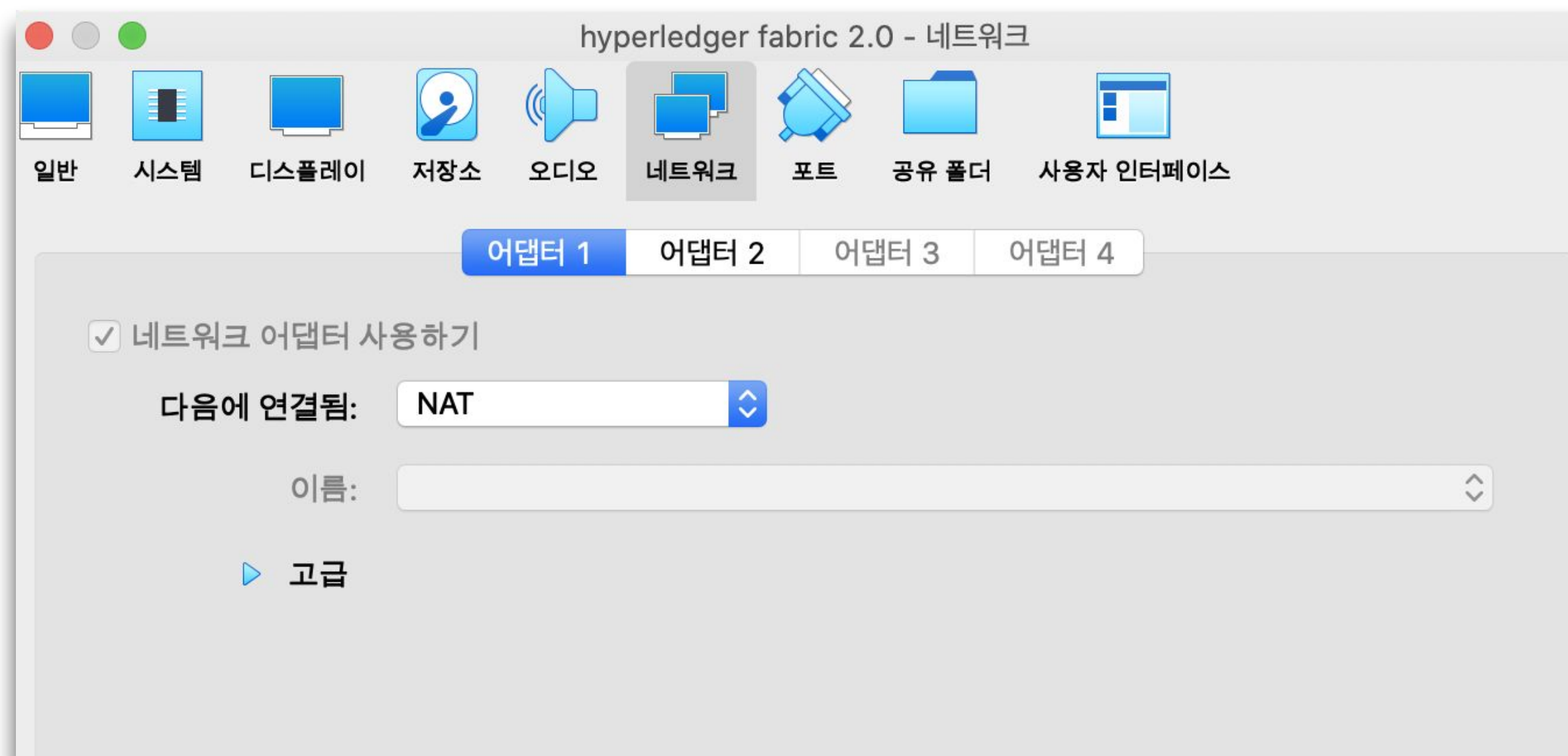
최고 주소 한계(U): 192.168.56.254

적용 Reset

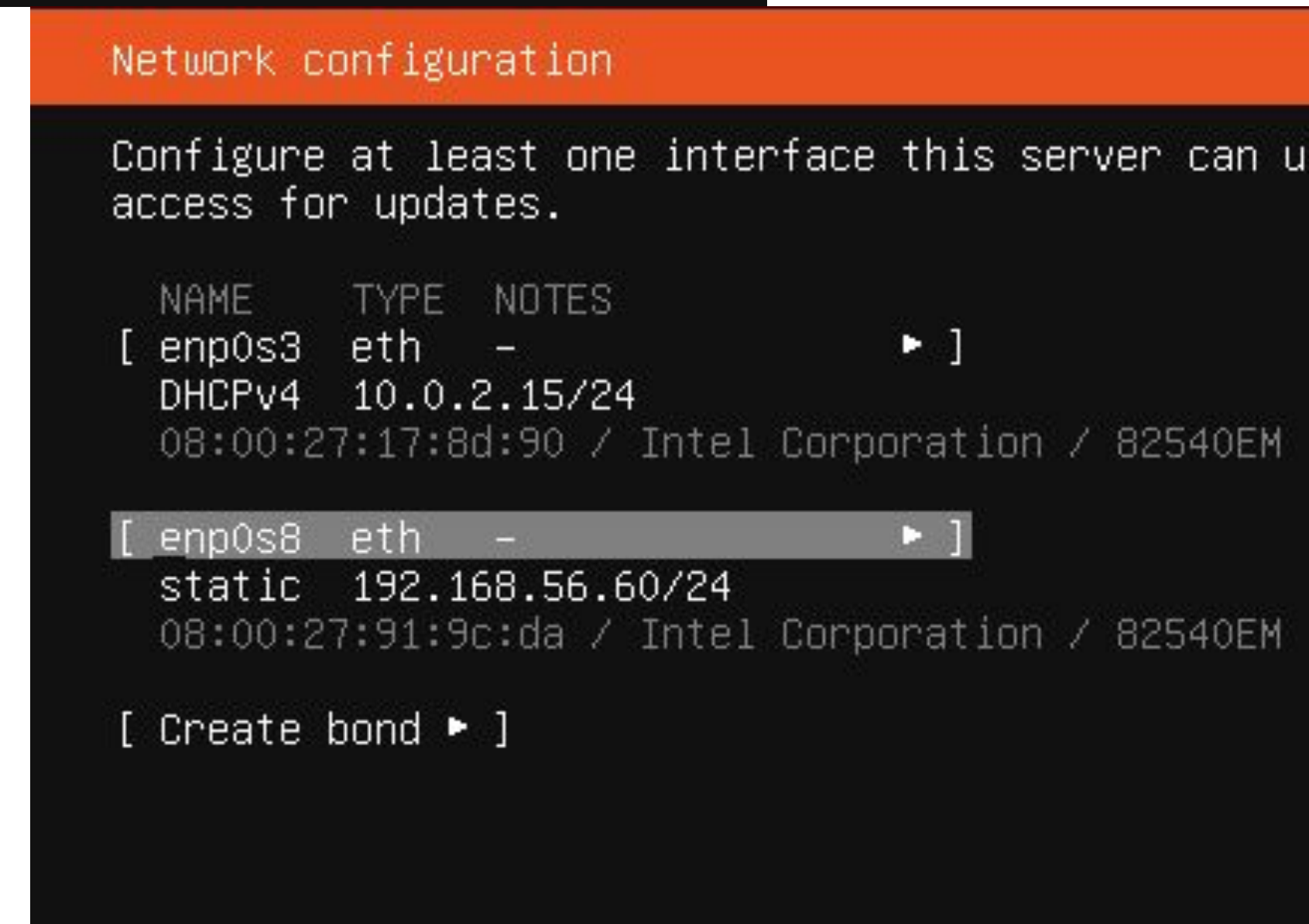
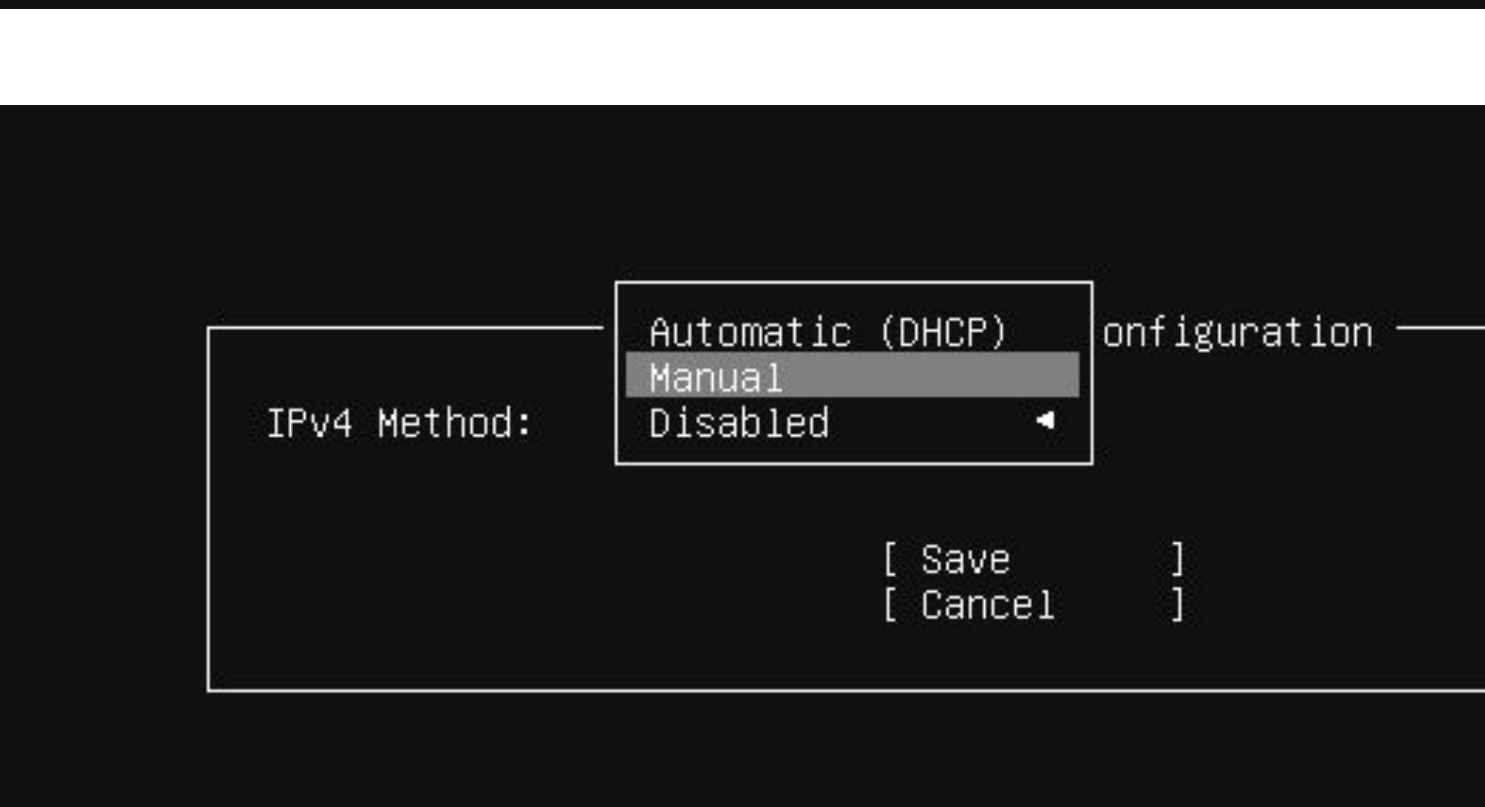
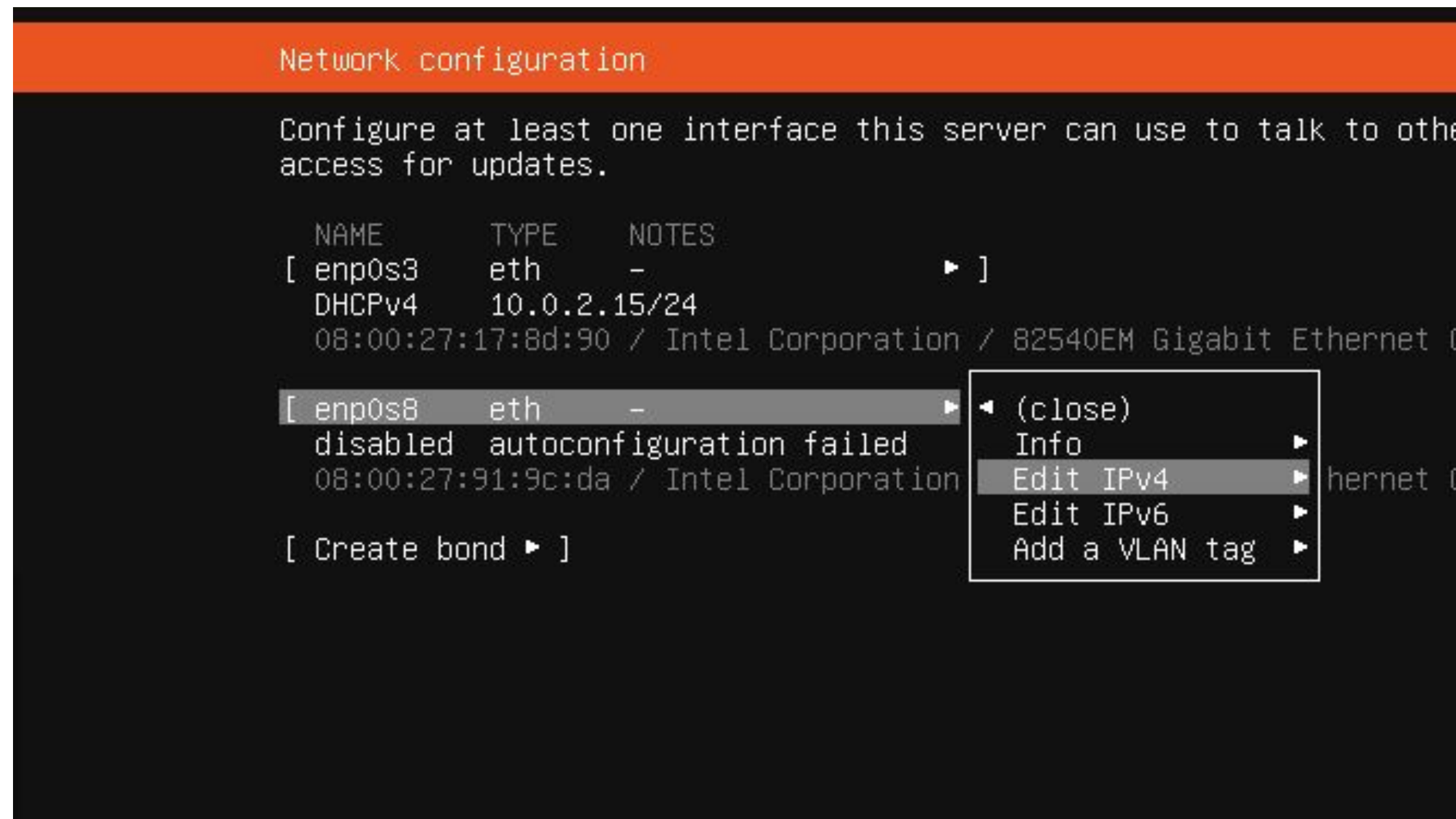
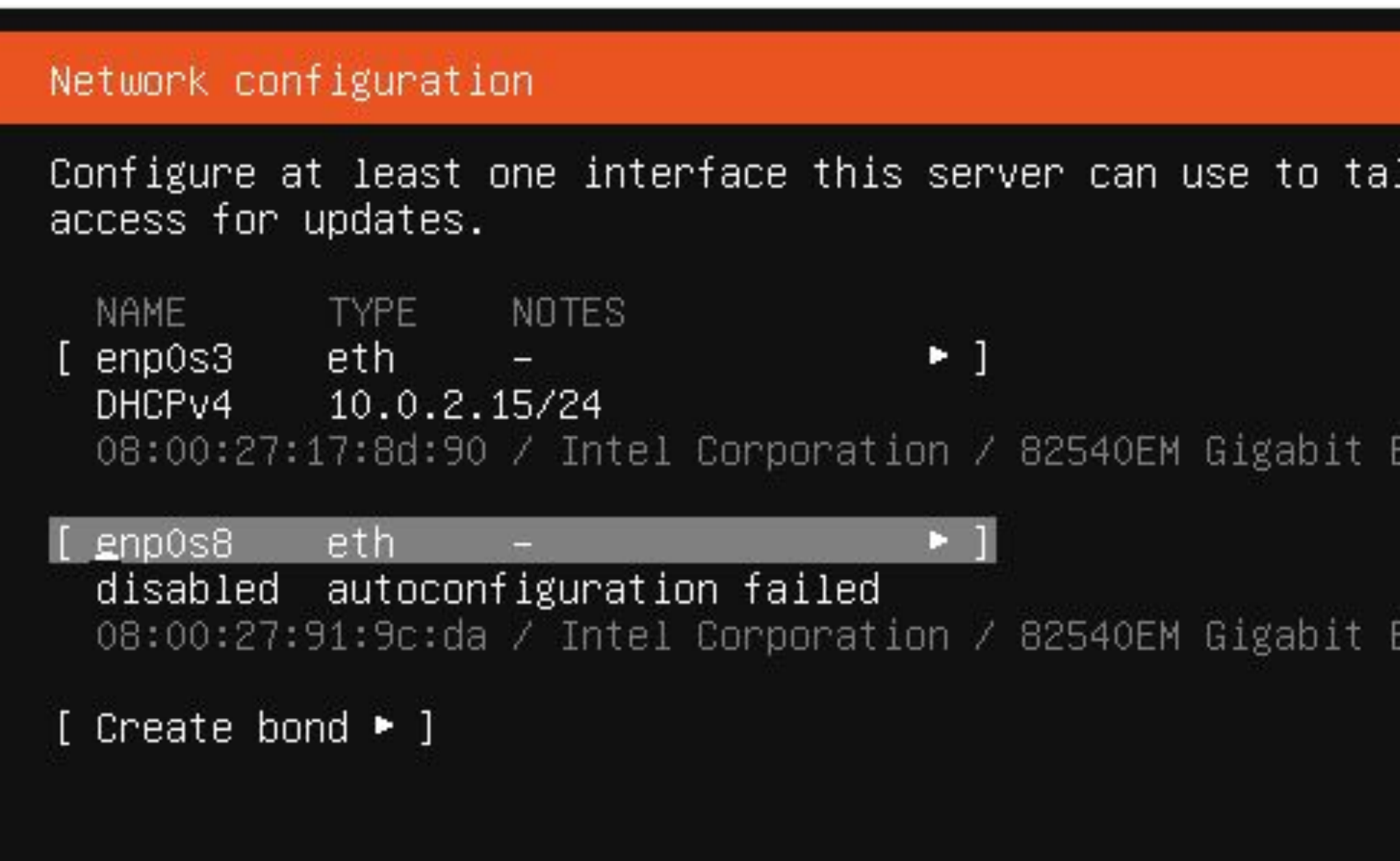
Network setup

In virtual box

- Adaptor 1 : NAT
- Adaptor 2: Host-Only Network



setup network info while installing



root 비밀번호 설정 및 계정 전환

- `sudo passwd root`
- `su - root`

Network setup

- check
vi /etc/netplan/50-cloud-init.yaml

Master

```
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses:
        - 192.168.56.60/24
      nameservers:
        addresses: []
        search: []
  version: 2
```

netplan apply

Reference :

<https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/#network-plugin-requirements>

Network setup

- vi /etc/sysctl.d/k8s.conf
- 리눅스 시스템에서 커널 파라미터
sysctl --system

```
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.ipv4.ip_forward = 1
```

net.bridge.bridge-nf-call-iptables = 1

net.bridge.bridge-nf-call-ip6tables = 1

net.ipv4.ip_forward = 1

Network setup

```
cat <<EOF | sudo tee  
/etc/modules-load.d/k8s.conf
```

```
overlay
```

```
br_netfilter
```

```
EOF
```

```
modprobe overlay
```

```
modprobe br_netfilter
```

모듈명	역할
overlay	컨테이너 이미지 계층 저장 방식 (OverlayFS 사용)
br_netfilter	브리지 네트워크의 트래픽이 iptables로 전달되도록 해줌 (쿠버네티스 네트워크 필수)

/etc/modules-load.d/ 디렉터리에 있으므로
systemd가

감지하여 다음 부팅 때 자동으로 모듈을 로딩

명령어	설명
modprobe overlay	overlay 커널 모듈을 로드하여, OverlayFS (파일시스템 계층화 기술)을 사용 가능하게 만듭니다. 컨테이너 이미지에서 필수입니다.
modprobe br_netfilter	br_netfilter 모듈을 로드하여, 브리지 네트워크에서 들어오는 트래픽을 iptables로 전달할 수 있게 만듭니다. 쿠버네티스 네트워킹에 필수입니다.

hostname

vi /etc/cloud/cloud.cfg

```
# This will cause the set+update hostname module to not operate (if true)
preserve_hostname: true
```

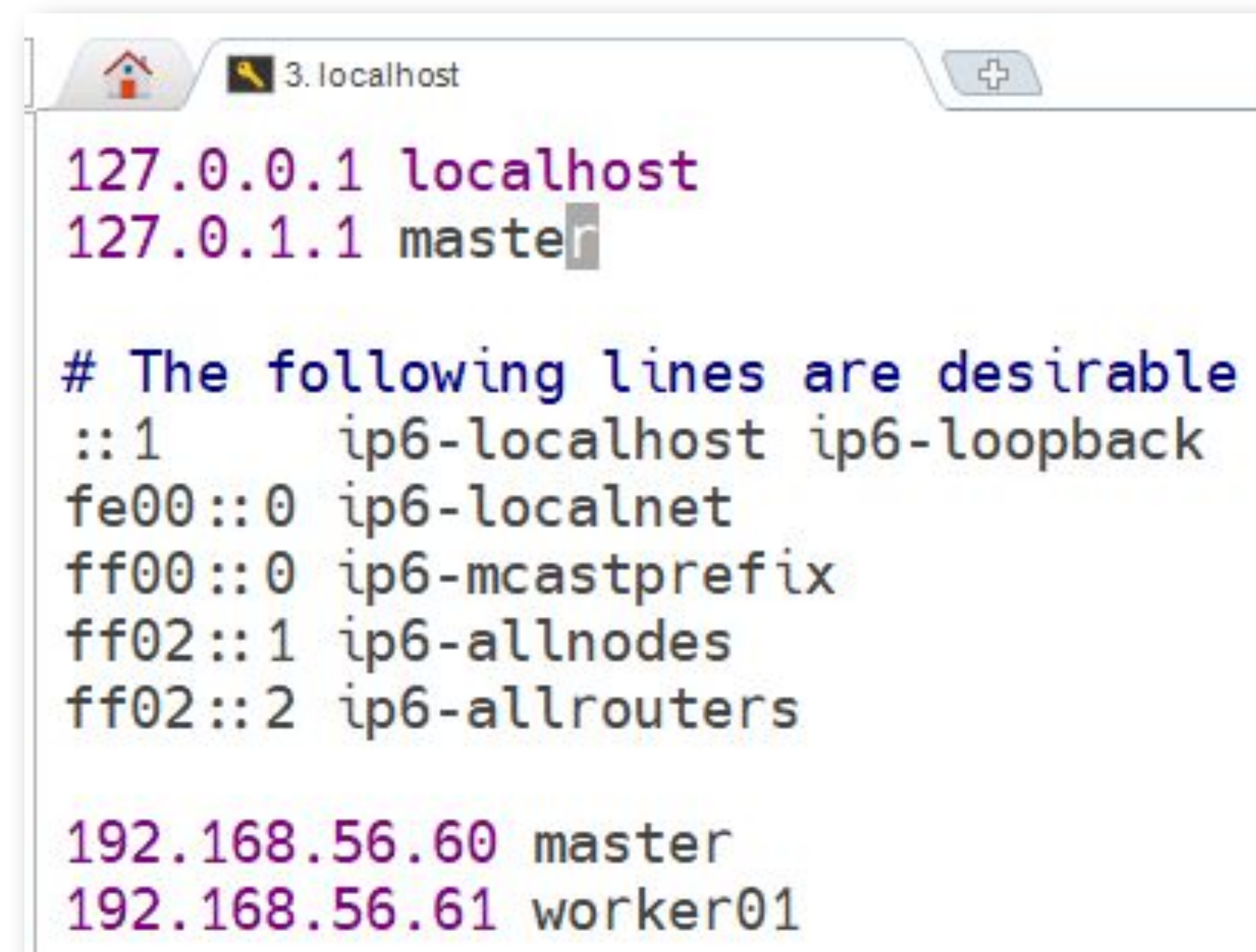
쿠버네티스 클러스터의 각 노드들은 고유한 호스트네임을 갖는 것이
중요

호스트네임이 변경되면 클러스터 내에서 노드를 식별하는 데 문제가
발생

systemctl restart systemd-logind.service

hosts

- vi /etc/hosts
192.168.56.60 master
192.168.56.61 worker01



```
127.0.0.1 localhost
127.0.1.1 master

# The following lines are desirable
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters

192.168.56.60 master
192.168.56.61 worker01
```


hostname setting

- Hostname setting in master node

```
hostnamectl set-hostname master
```

- If worker's hostname is equal to master, then you will see following error message on worker node when executing kubeadm join
>> a Node with name ** and status "Ready" already exists in the cluster.

Requirements

Docker

- Memory swap off

swapoff -a

- vi /etc/fstab

```
UUID=a4241556-c44e-4b66-8a52-4a82994c2c99 / ext4 defaults 0 0
#/swap.img none swap sw 0 0
```

```
kopo@master:~$ free -m
              total        used        free
Mem:           2980          236         2083
Swap:            0            0            0
```

재부팅 후 다시 swap이 켜지지 않도록” 막기 위한 조치

메모리 스왑(Swap)이란?

RAM이 부족할 때, 하드디스크(또는 SSD)의 일부 공간을 가상 메모리처럼 사용하는 기능입니다.

운영체제는 실행 중인 프로그램과 데이터를 주기억장치(RAM)에 적재합니다.

하지만 RAM이 부족해지면, 시스템은 덜 사용되는 메모리 데이터를 디스크로 옮겨서 RAM을 확보합니다.

이때 옮겨지는 공간이 바로 **스왑 영역(swap space)**입니다.

VM 복제

가상 머신 선택

New Machine Name and Path

이름(N): K8S Worker01 ✓

경로(P): C:\Users\whwy\VirtualBox VMs

Clone Type

☒ 완전한 복제(F)

☐ 연결된 복제(L)

스냅샷

☐ Current Machine State

☐ 모두(E)

추가 옵션

MAC 주소 정책(O): 모든 네트워크 어댑터의 새 MAC 주소 생성

추가 옵션: ☐ 디스크 이름 유지하기(D)

☐ 하드웨어 UUID 유지(W)

도움말(H) 이전(B) 완료(F) 취소(C)

VM 복제

worker node

- check
vi /etc/netplan/50-cloud-init.yaml

```
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses:
        - 192.168.56.61/24
      nameservers:
        addresses: []
        search: []
  version: 2
~
```

netplan
apply

hostname setting

- Hostname setting in worker node

```
hostnamectl set-hostname worker01
```

- If worker's hostname is equal to master, then you will see following error message on worker node when executing kubeadm join
>> a Node with name ** and status "Ready" already exists in the cluster.

Port Open

master(control plane) node

```
ufw allow "OpenSSH"

ufw enable

ufw allow 6443/tcp

ufw allow 2379:2380/tcp

ufw allow 10250/tcp

ufw allow 10259/tcp

ufw allow 10257/tcp

ufw status
```

Worker node

```
ufw allow "OpenSSH"

ufw enable

ufw status

ufw allow 10250/tcp

ufw allow 30000:32767/tcp

ufw status
```

<https://kubernetes.io/docs/reference/networking/ports-and-protocols/>

Control plane [↗](#)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443	Kubernetes API server	All
TCP	Inbound	2379-2380	etcd server client API	kube-apiserver, etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10259	kube-scheduler	Self
TCP	Inbound	10257	kube-controller-manager	Self

Although etcd ports are included in control plane section, you can also host your own etcd cluster externally or on custom ports.

Worker node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	30000-32767	NodePort Servicest	All

[tip] virtual box range port forwarding

1. Vbox register(If needed) : VBoxManage registervm /Users/sf29/VirtualBox\ VMs/Worker/Worker.vbox
2. for i in {30000..30767}; do VBoxManage modifyvm "Worker" --natpf1 "tcp-port\$i,tcp,, \$i,, \$i"; done

Kubernetes 설치

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Kubernetes setup

Install containerd

```
apt update
apt install -y containerd
containerd --version
```

```
# containerd 설정 파일 생성
```

```
mkdir -p /etc/containerd
```

```
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
```

```
# 설정 변경
```

```
vi /etc/containerd/config.toml
```

```
SystemdCgroup = true
```

```
# containerd 재시작
```

```
systemctl restart containerd
```

```
# 부팅 시 자동 실행
```

```
systemctl enable containerd
```

```
# 상태 확인
```

```
systemctl is-enabled containerd
```

```
systemctl status containerd
```


Kubernetes setup

Install kubernetes

```
apt install apt-transport-https ca-certificates curl -y
```

```
# curl -fsSLo /usr/share/keyrings/kubernetes-archive-keyring.gpg https://packages.cloud.google.com/apt/doc/apt-key.gpg
```

```
curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key --keyring /usr/share/keyrings/cloud.google.gpg add -
```

더 이상 유효하지 않음

```
#echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-keyring.gpg] https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://apt.kubernetes.io/kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
apt update
```

```
apt install kubelet kubeadm kubectl
```

```
apt-mark hold kubelet kubeadm kubectl ← # 패키지가 자동으로 업그레이드 되지 않도록 고정
```

기본 도구 및 인증서 설치

구글 클라우드의 GPG 키 추가

쿠버네티스 APT 저장소 추가

패키지 목록 업데이트

쿠버네티스의 주요 구성 요소 설치

설치된 패키지 고정

```
kubeadm version https://littlemobs.com/blog/kubernetes-package-repository-deprecation/
```

```
kubelet --version
```

```
kubectl version
```

<https://kubernetes.io/ko/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Kubernetes setup

Install kubernetes

```
apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.36/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.36/deb/ ' | sudo tee /etc/apt/sources.list.d/kubernetes.list

apt-get update

apt-get install -y kubelet kubeadm kubectl

apt-mark hold kubelet kubeadm kubectl
```

```
kubeadm version
```

```
kubelet --version
```

```
kubectl version
```

기본 도구 및 인증서 설치

구글 클라우드의 **GPG** 키 추가

쿠버네티스 **APT** 저장소 추가

패키지 목록 업데이트

쿠버네티스의 주요 구성 요소
설치

설치된 패키지 고정

CNI(컨테이너 네트워크 인터페이스) 플러그인 설치

```
mkdir -p /opt/bin/
```

```
curl -fsSLo /opt/bin/flanneld
```

```
https://github.com/flannel-io/flannel/releases/download/v0.19.0/flanneld-amd64
```

```
chmod +x /opt/bin/flanneld
```

```
lsmod | grep br_netfilter
```

```
kubeadm config images pull
```

더 이상 유효하지 않음

Master node setting

Kubeadm init

```
kubeadm init --pod-network-cidr=10.244.0.0/16 \  
--apiserver-advertise-address=192.168.56.60 \  
--cri-socket=unix:///run/containerd/containerd.sock
```

Your ip address

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

kubectl 명령을 현재 로그인한 일반 사용자 계정에서 바로 사용

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 192.168.56.60:6443 --token arh6up.usqi6daj82rj4rg2 \  
--discovery-token-ca-cert-hash sha256:12035aced64146fc7ccc5e3e737192c7209bc6bacc3fdb5b14400f6f9fd9adfa
```

worker01에서
수행

```
root@master:/home/kopo# kubectl get nodes  
NAME        STATUS    ROLES    AGE   VERSION  
master      NotReady control-plane 7m10s v1.32.3  
worker01    NotReady <none>    16s   v1.32.3
```

master에서 수행(다음 장 명령 실행 후)

Reference :

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/>

Master node setting

```
mkdir -p $HOME/.kube
```

```
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
chown $(id -u):$(id -g) $HOME/.kube/config
```

```
kubectl cluster-info
```

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
```

```
kubectl get pods --all-namespaces
```

```
kubectl get nodes -o wide
```

```
curl -k https://localhost:6443/version
```

[Reference :](#)

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/>

Worker Node Join

더 이상 유효하지 않음

```
kubeadm join 192.168.56.60:6443 --token arh6up.usqi6daj82rj4rg2 \
--discovery-token-ca-cert-hash
sha256:12035aced64146fc7ccc5e3e737192c7209bc6bacc3fdb5b14400f6f9fd9adfa
```

master
node:

```
root@master:~/.kube# kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-flannel	kube-flannel-ds-45m24	1/1	Running	0	56s
kube-flannel	kube-flannel-ds-cjbnw	1/1	Running	0	3m26s
kube-system	coredns-5d78c9869d-55k6l	1/1	Running	0	6m53s
kube-system	coredns-5d78c9869d-ks4sj	1/1	Running	0	6m53s
kube-system	etcd-master	1/1	Running	0	7m6s
kube-system	kube-apiserver-master	1/1	Running	0	7m8s
kube-system	kube-controller-manager-master	1/1	Running	0	7m6s
kube-system	kube-proxy-6bddb	1/1	Running	0	56s
kube-system	kube-proxy-hgcdq	1/1	Running	0	6m53s
kube-system	kube-scheduler-master	1/1	Running	0	7m6s

```
root@master:~/.kube# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	8m	v1.27.2
worker01	Ready	<none>	107s	v1.27.2

kubectl get pods --all-namespaces

kubectl get nodes -o wide

curl -k

<https://localhost:6443/version>

Kubectl Autocomplete

k = kubectl

Kubectl autocomplete

BASH

```
source <(kubectl completion bash) # set up autocomplete in bash into the current shell, bash-completion package should be installed first.
echo "source <(kubectl completion bash)" >> ~/.bashrc # add autocomplete permanently to your bash shell.
```

source <(kubectl completion bash) # set up autocomplete in bash into the current shell, bash-completion package should be installed first.

You can also use a shorthand alias for kubectl that also works with completion:

echo "source <(kubectl completion bash)" >> ~/.bashrc # add autocomplete permanently to your bash shell.

```
echo "alias k=kubectl" >> ~/.bashrc
```

```
echo "complete -o default -F __start_kubectl k" >>
```

```
~/.bashrc
```

```
# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
  . /etc/bash_completion
fi
source <(kubectl completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k
```

source

Source : <https://kubernetes.io/docs/reference/kubectl/cheatsheet/>

bash-completion 로드

```
apt install -y bash-completion
```

```
source /usr/share/bash-completion/bash_completion
```

~/.bashrc 하단에 아래와 같이 편집

```
# bash completion
```

```
if [ -f /usr/share/bash-completion/bash_completion ]; then
```

```
    source /usr/share/bash-completion/bash_completion
```

```
fi
```

```
# kubectl completion
```

```
source <(kubectl completion bash)
```

```
# kubectl alias
```

```
alias k=kubectl
```

```
complete -o default -F __start_kubectl k
```


Hello world

hello world

- Master node : deploy pod

```
root@kopo:~# kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
deployment.apps/kubernetes-bootcamp created
root@kopo:~# k get deployments.apps
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
kubernetes-bootcamp 0/1    1           0          12s
root@kopo:~# k get deployments.apps
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
kubernetes-bootcamp 1/1    1           1          19s
root@kopo:~# k get pods -o wide
NAME                                READY  STATUS    RESTARTS  AGE  IP          NODE     NOMINATED NODE  READINESS GATES
kubernetes-bootcamp-6f6656d949-sdhqp 1/1    Running   0          35s  10.244.1.2  worker01 <none>         <none>
```

- Worker node : check the pod is running

```
root@worker01:~# curl http://10.244.1.2:8080
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-6f6656d949-sdhqp | v=1
```

Flannel Pod 네트워크 통신 오류

Flannel Pod 네트워크 통신 오류

증상

- Worker Node에서는 Pod IP(10.244.1.2) 접속 가능
- Control Plane에서는 해당 Pod IP 접속 불가

원인

- VirtualBox VM에 NAT와 Host-Only NIC가 함께 존재
- Flannel이 노드 간 통신용 NIC를 잘못 선택

enp0s3 → 10.0.2.15 ❌ NAT, Flannel이 자동 선택
enp0s8 → 192.168.56.x ✅ Host-Only, 노드 간 통신용



로그에서 확인:

Using interface with name enp0s3 and address 10.0.2.15
PublicIP: 10.0.2.15



kubectl edit daemonset kube-flannel-ds -n kube-flannel

아래 빨간색 부분 추가

args:
- --ip-masq
- --kube-subnet-mgr
- **--iface=enp0s8**

Terminology and concept

get

kubectl get all

kubectl get nodes

kubectl get nodes -o wide

kubectl get nodes -o yaml

kubectl get nodes -o json

describe

kubectl describe node <node name>

kubectl describe node/<node name>

Other

kubectl exec -it <POD_NAME>

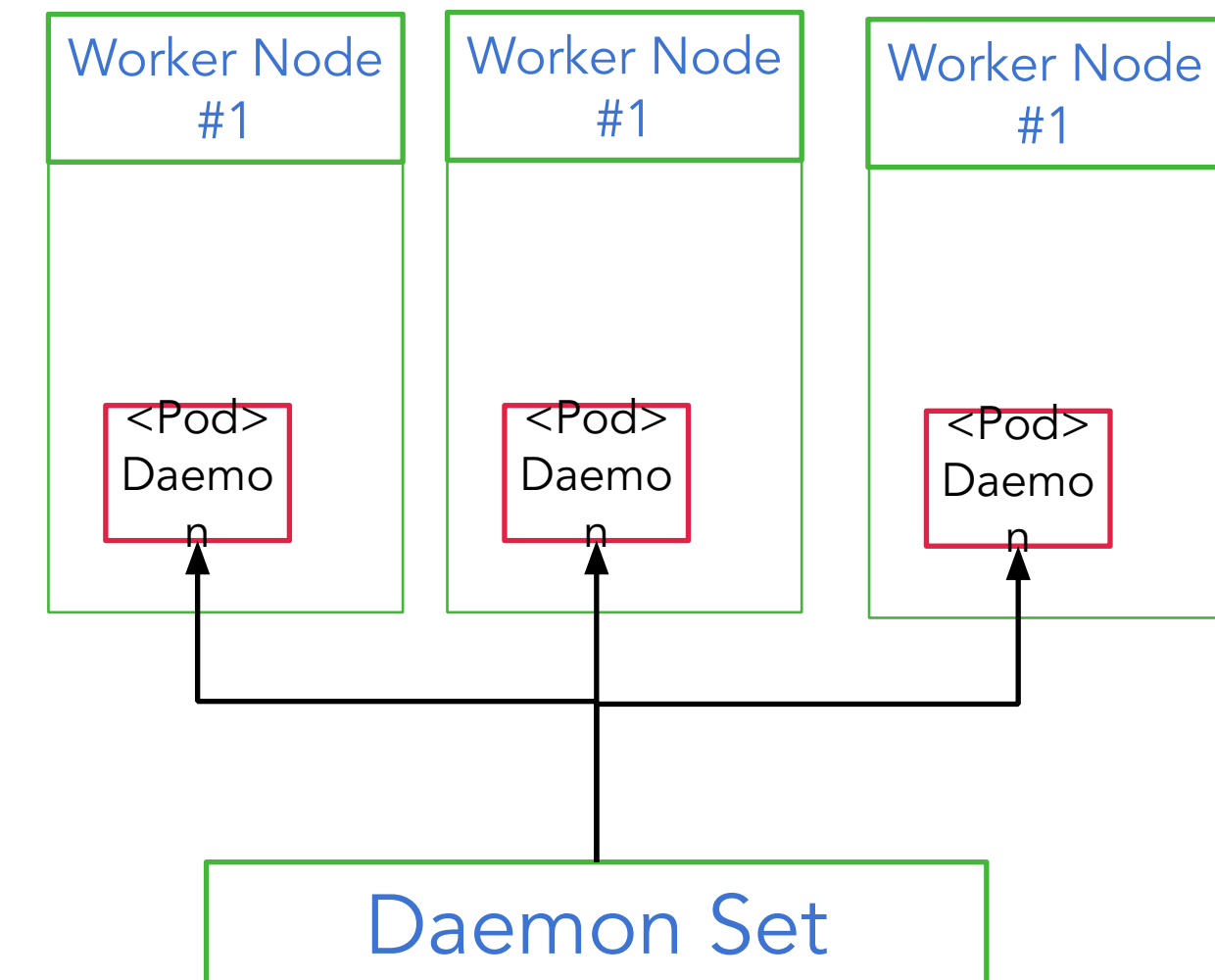
kubectl logs <POD_NAME|TYPE/NAME>

kubectl apply -f <FILENAME>

kubectl delete -f <FILENAME>

Deployment method

- Daemon set : only single container per each node
예: 로그 수집기, 모니터링 에이전트
- Replica set : control a number of container
예: 웹 서버 여러 개 실행
- Stateful set : replica set + control sequence
예: 데이터베이스 클러스터(MySQL, Kafka)



Kubectl practice

<https://github.com/wonyongHwang/kopoK8sPractice.git>

Make a pod

One container / one pod

- k apply -f first-deploy.yml

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: kopotest
```

```
  labels:
```

```
    type: app
```

```
spec:
```

```
  containers:
```

```
  - name: app
```

```
    image: nginx:latest
```

- k get po
k get all
k describe po/kopotest

Liveness/Readiness probe

○ Liveness probe : check after boot up

- 목적: 컨테이너가 살아있는지(동작 중인지) 확인
- 문제 발생 시: 실패하면 컨테이너를 재시작함

○ Readiness probe : check before boot up

- 목적: 컨테이너가 트래픽을 받을 준비가 되었는지 확인
- 문제 발생 시: 실패하면 **Service**에서 제외시켜 트래픽을 받지 않게 함

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
```

◆ initialDelaySeconds: 10

- 컨테이너가 시작된 후, 몇 초 뒤에 처음 **probe**를 실행할지 설정
- 예: 컨테이너가 시작되자마자 체크하는 것이 아니라, 10초 후에 처음으로 **health check** 수행

◆ periodSeconds: 5

- **probe**를 주기적으로 수행할 간격(초 단위)
- 예: 처음 probe가 실행된 후부터 5초마다 다음 probe가 실행됨

Make a pod

One container / one pod

k apply -f first-deploy-lp.yml
k apply -f first-deploy-hc.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: kopotest-lp
  labels:
    type: app
spec:
  containers:
  - name: app
    image: nginx:latest
    livenessProbe:
      httpGet:
        path: /not/exists
        port: 80
      initialDelaySeconds: 5
      timeoutSeconds: 2 # Default 1
      periodSeconds: 5 # Defaults 10
      failureThreshold: 1 # Defaults 3
```

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
  timeoutSeconds: 2
  failureThreshold: 3
  successThreshold: 2
```

- 컨테이너 시작 후 5초 뒤에 probe 시작
- 3초마다 체크
- 응답이 2초 넘게 걸리면 실패
- 3번 연속 실패해야 실제 실패로 간주
- 2번 연속 성공해야 준비 완료로 간주

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
kopotest-lp	1/1	Running	4 (33s ago)	73s	10.244.1.8	worker01	<none>	<none>

Make a pod

Multi container / one pod

- K apply -f multi-container.yml

```
root@kopo:~/project/ex02# k get pod
NAME                                READY STATUS RESTARTS AGE
kubernetes-bootcamp-6f6656d949-sdhqp 1/1   Running 0      10h
two-containers                       1/2   NotReady 0      34m
root@kopo:~/project/ex02# k logs po/two-containers
error: a container name must be specified for pod two-containers, choose one of: [nginx-container
debian-container]
root@kopo:~/project/ex02# k logs po/two-containers nginx-container
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
127.0.0.1 - - [23/Jul/2020:12:52:56 +0000] "GET / HTTP/1.1" 200 42 "-" "curl/7.64.0" "-"
```

```
root@master:~/kopoK8sPractice/Practice/ex02# more multi-container.yml
apiVersion: v1
kind: Pod
metadata:
  name: two-containers
spec:

  restartPolicy: Never

  volumes:
  - name: shared-data
    emptyDir: {}

  containers:

  - name: nginx-container
    image: nginx
    volumeMounts:
    - name: shared-data
      mountPath: /usr/share/nginx/html

  - name: debian-container
    image: debian
    volumeMounts:
    - name: shared-data
      mountPath: /pod-data
    command: ["/bin/sh"]
    args: ["-c", "echo debian 컨테이너에서 안녕하세요 > /pod-data/index.html"]
```

Make a pod

Multi container / one pod

```
k exec -it two-containers -c nginx-container -- /bin/bash
```

```
cd /usr/share/nginx/html/
```

```
more index.html
```

블룸의 종류 참조 :
<https://bcho.tistory.com/1259>

You can see that the debian Container has terminated, and the nginx Container is still running.

Get a shell to nginx Container:

```
kubectl exec -it two-containers -c nginx-container -- /bin/bash
```

In your shell, verify that nginx is running:

```
root@two-containers:/# apt-get update
root@two-containers:/# apt-get install curl procps
root@two-containers:/# ps aux
```

```
Do this work-around if dns error occurs...
root@two-containers:/# cat > etc/resolv.conf <<EOF
> nameserver 8.8.8.8
> EOF
```

The output is similar to this:

USER	PID	...	STAT	START	TIME	COMMAND
root	1	...	Ss	21:12	0:00	nginx: master process nginx -g daemon off;

Recall that the debian Container created the `index.html` file in the nginx root directory. Use `curl` to send a GET request to the nginx server:

```
root@two-containers:/# curl localhost
```

The output shows that nginx serves a web page written by the debian container:

```
Hello from the debian container
```

Source : <https://kubernetes.io/docs/tasks/access-application-cluster/communicate-containers-same-pod-shared-volume/>

LENS 설치

- <https://k8slens.dev/>

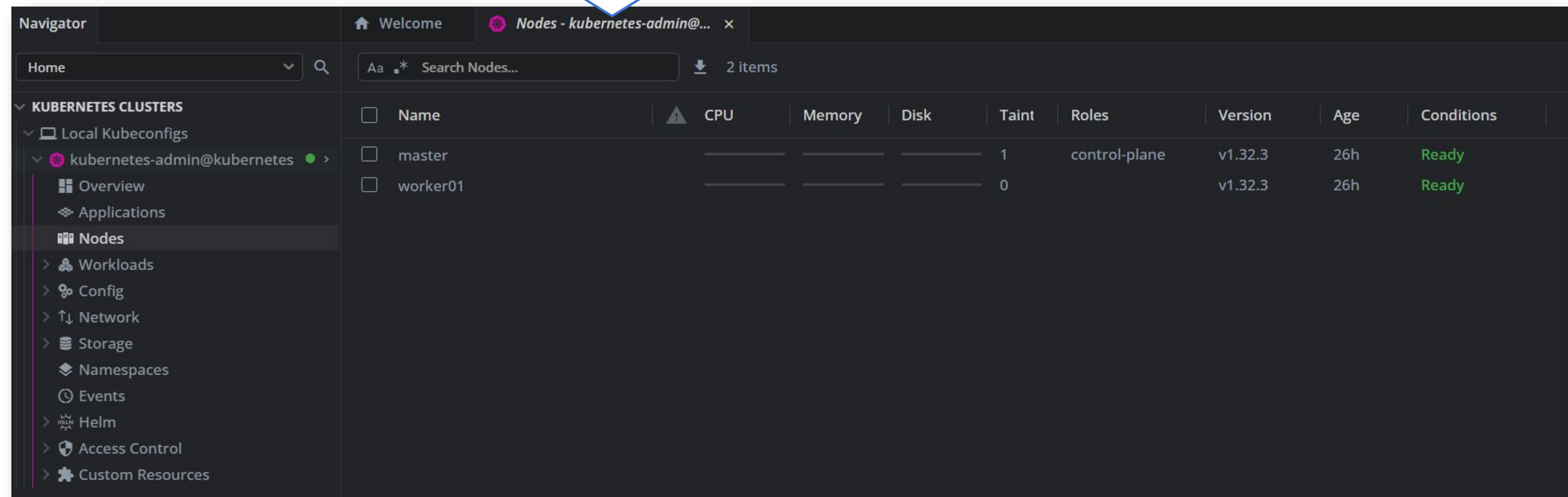
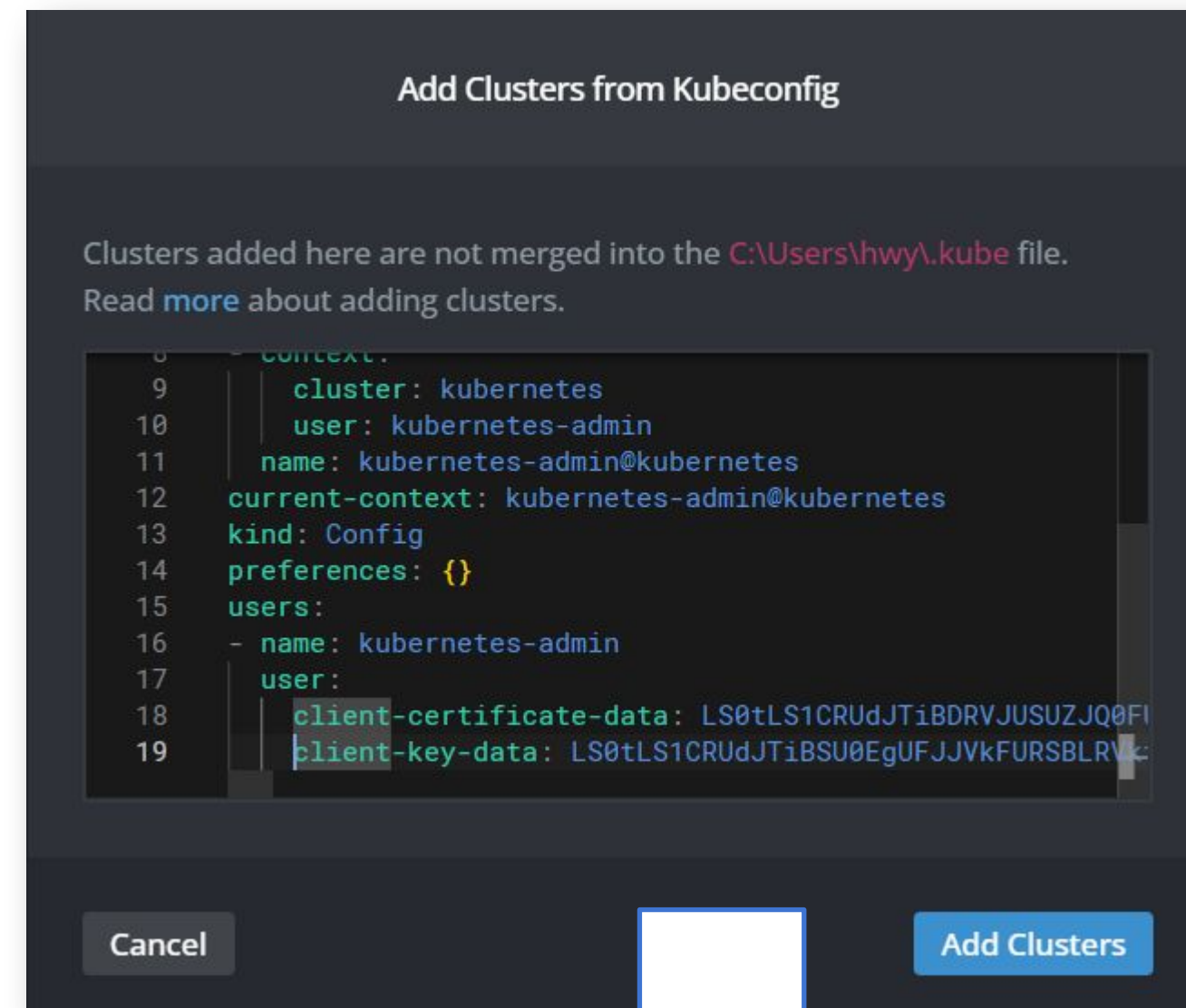
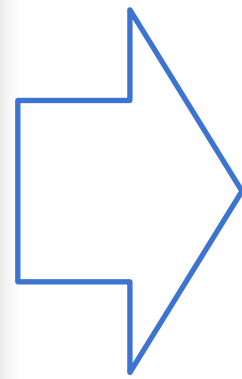
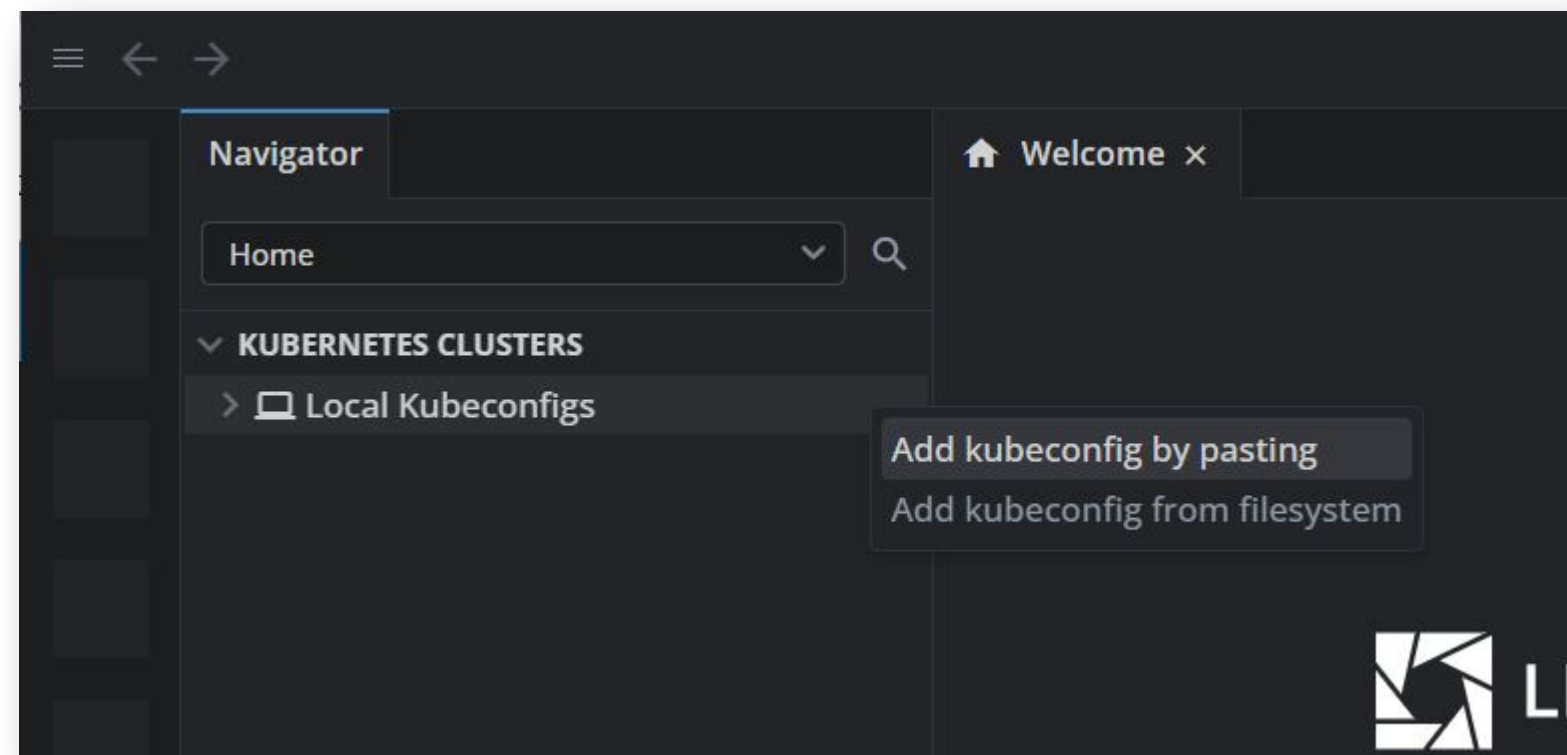


LENS 설치

- kubectl config view --minify --raw

```
root@master:~/kopoK8sPractice/Practice/ex02# kubectl config view --minify --raw
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tck1JSURCVENDQWUyZ0F3SUJBZ0lJJulQy
    ktvWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQXhNS2EzVmIawEp1WlHsbGN6QWVGdzB5TlRBME1UY3hNVE0zTWps
    E1UUXlNamxhTUJVeApFekFSQmd0VkJBTVRDbXQxwW1WeWJtVjBawE13Z2dFau1BMEdDU3FHU0lIM0RRRUJBUVVBQTRJ
    Z0VLckFvSUJBURtTmJEWlPvMlgzRFpZSs2S2VDVm1PNC93T1N5K1RpdDJwWdNP0Vdmenc0V0dlbXBZeXoxRFJwU1UKbHdCMXVQ
    aUlJSExmc2VoeFE2a1A1cjVzY25uL2hzbEd5cXZlM0V0S05LR0pXeU5Md0dVS2h5QXBZ0Qp4L1UrbXZiNHBQaUdiQSs4NlM3Q0Rl
    YzYxQ3JzUWlDWEhqV0tBS2hRcnlyODlvQTFTaEN4ZnJxMHZwClB6clNnS01iUUVSZ1ZlRXhmVmHEbmRFbVN6dU0zbW9MNUZqNGlJ
    RURmbFBGcDhCYW51V0R0QjgzNlYKbkNudXBidVFlVmhxUElsWmw0NGVCWUFxT3l2SHAvR2gvQmljMmpndTRYaXl4aUFI4Y8wQWhf
    RDRuLwo4Qjd5NwNwM0l6dGNnZEREQTLmb0RHSkxENlJ6QWdNqkFBR2pXVEJYTUE0R0ExVWREd0VCL3dRRUF3SUNwREFQckJnTlZl
    RUJUQURBUEUgVtUIwR0ExVWREZ1FXQkJSRjZ0Vvc3ZwL2YXpqZnZYMWFDdkJhcmJuSFR6QVYKQmd0VkhSRUVEakFNZ2dwcmlRZXmx
    ek1BMEdDU3FHU0lIM0RRRUJDd1VBQTRJQkFRQUJRRkUxNHBhNwpPY3c0MVI2bk9tdUVuazA0RwPSNGI3VFZXTk10UXV4YVNOYm95
    ckhvZGZHYlJlZ0hIcWQwVUFHREpFcIorN1hFb2hYOGJtSnRjOGJYQ3NjNytMlkl2TEJoUWQ5YUp3Tm8xvjdesUN1N0pKeEd0L1N0
    bnY3VE4KRWJsM0Q1YzhVREl5eHF2bWxnV3dvSENfNlVBFY2hCb1ZtOGNyS2xrVHB0S2RxbXBWTVqSU5SQUtQcThKMhdUUApzejV
    T2FSTE9NcnZTLy9hUjZkRFFVb0FXeGxSWmZkZlpiaTlDZ3VDWk1LRm55VkFrdVdSTnFIWEMwCkd2SjI4a09RRUJDU1Rad0Q4Ww9Y
    VmNyYzVUUHVswk1Pamc0RlJpUEgyVTZZWlFHY2JrQ0dYMUphb3oKMzUvVXN3anpiekRpCi0tLS0tRU5EIEFNUlRJRklDQVRFLS0t
    server: https://192.168.56.60:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tck1JSURLVENDQWhHZ0F3SUJBZ0lJV0MxaVl
    3RFFZSkvWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQXhNS2EzVmIawEp1WlHsbGN6QWVGdzB5TlRBME1UY3hNVE0zTWpsYU2
    wTVRjeE1UUXlNamxhTUR3eApIekFkQmd0VkJBB1RGbXQxwW1WaFpHMDZZMngxYzNSbGNpMwhaRzFwYm5NeEdUQVhCZ05WQkFNVEV
    5CmJtVjBawE10WVdSdGFxNHdnZ0VpTUEwR0NtCUdTSWIZrFFFQkFRVUFBNElCRHdBd2dnRUtBb0lCQVFER01jb2MKK0lzadDKSh
    DMGp1bVZNWFD3cXBIQmJYUjlbQmRmLyt0ZlhoRGdVU0tPcjgxdUNWS01YZHd5QStoMapXUzVtZW50cGY1dkF4UULYSRHNTVYM3M
    rej13RFZBWFRpU3VidDNUYUVCZzVhM3FHaW82OVFSclJpckllWxppdnVoZ1hsYXdYcVhLaWw3UjVvVaksxd3pLR0ZpdHJlOXZGa0J
    tTnhIV3BrQXJkVWFwam90bVgKWUtTT0lReXZiZUI1eVpDcFRw0HNkaxh5MG5IU1hKZ2pQTMfTVFc4RTdEVFp4aDEvamVT0WhxcmY
    waAp1bUpXM3I2M1JXTE1GbWfwdFMzblp4bmVRWVY1bWtkcHZZcFBsRGxaaUR0cDBhMF1hb1NZR1pKcXc10TNkaFdScmV1N2ZZaV
    JRDdBZ01CQUFHAlZqQlVNQTRHQTfVZER3RUIvd1FFQXdJRm9EQVRCZ05WSFNVRUREQUSKQmdnckJnRUZCUWNEQWpBTUJnTlZlUk1
    qQUFNQjhhQTfVZE13Uv1NQMfBRkFubzFSYnQ2S2lyT04r0QpmVm9LOEzxdHVjZFBnQTbHQ1Nxr1NjYjNEUUVcQ3dVQUE0SUJBUR
    3ZjdRazB0REZ1dXdFZm5RbytkCjVMUmhhSwpjQnZycng4bDFTdEFqZ0RtS2VoYXN6SkJuZjIyTzBnTjdtZFVaeDFaVvPWmkxCe1
    hNDYKTnBjUXRTQmZHV21TQlg1OG8yK0xiMUprdjIrVW9BSFGwN1o3a2FCY0J3S1pmZXBvYlYzbXJNeWRjVVZhdjIySQp6cm1BU3E
    wk0Fib05ldXB1K2Urc1NEVlQrRWJGU0Y5NURYRmZTNpVdZDQ3d0RyRmZTNpVdZDQ3d0RyRmZTNpVdZDQ3d0RyRmZTNpVdZDQ3d0
    OU3Bnbk5VZ1U5RmVHNThtVENHTXRSnBVUE5sYVFMZEH4ZSsKMnMTW5ksnl4MwxEQlViUXdMVlRtYlY1qjMwBEZhSjg0evhXcUp
    s0Fg2S1BQVjFwVkhFwkluCi0tLS0tRU5EIEFNUlRJRklDQVRFLS0tLS0K
    client-key-data: LS0tLS1CRUdJTiBSU0EgUUFJJVkFURSBLRVktLS0tLQpNSU1Fb3dJQkFBS0NBUEUvBeGpIS0hQaUxJZXl
    4NEF0STdwbFRGMXNlcVJ3VzEwZlFBWFgvL2pYMTRRcjRGRWlqcS90YmdxU2pGM2NNZ1BvZEZrdVpucHdxWctid01VQ0Z5VXh1ZVY
    aTXBNL2NBMVFGMDRrcm0KN2QwMmhBWU9XddZob3FPdlVfYTBAuhtTTRYn29ZrjVXc0Y2bHlvcGUwZVZJeXJrTXloaFlyYXl2Yn
    BWENKamNSMXFaQUszVkdSWTZEwmyQ2tqaUVNcjIzZ2VjbVFXvTmTmEhZc2N0SngwbHlZSxp6V2trMXZCT3cwCjJjWWRmNDNrdl
    BWFc5SVhwaVZ0Nit0MFZpekJabXFivXQ1MmNam2tHRmVacEhhYjJLVDVRNVdZZzcKYWRHdEdHcDBtQm1TYXNPZmQzWVZrWHJ1Mz
    kRUNBK3dJREFRQUJBb0lCQUVASTE5MElRd2JueHlwdgphbGh10Ww4Um8wSTJSUDgrcV2RGJpRDZERkFPQmNzSEo4TDA0dVZvOE5
    iUGdMckw20TlaZC82ajRoCnM5RmJ6QTc3TUxRMDd6L3I0RTE3UDR4ZkZ0dHk1Y2x1T0pDMz15SkFwNHlRRVJBBrkxpR3d3VDFmdW
    KNFNGUllniSS8wSEFEZaFlud1VZVU8xK0nTMMZZd21nSUVwBDVjTEFEuam9ZBVMPDd1pcUtuUXVkd0FE5UkVKSERNBZ0ppNExGR2d6Sm
```


LENS 설치



LENS 설치

The screenshot displays the Kubernetes Lens application interface. The left sidebar shows the navigation menu with categories like KUBERNETES CLUSTERS, Local Kubeconfigs, and Workloads. The main area shows the 'Workloads Overview' for the 'default' namespace. It features a dashboard with circular progress indicators for Pods (1), Deployments (0), Daemon Sets (0), Stateful Sets (0), and Replica Sets (0). Below this, there are sections for Jobs (0) and Cron Jobs (0). A table at the bottom lists the running pods, with one pod named 'two-containers' in the 'default' namespace, running on the 'worker01' node. A red box highlights the context menu for the 'two-containers' pod, which includes options like Attach Pod, Shell, Evict, Share link..., Logs, Edit, and Delete.

Pod List Table:

Name	Namespace	Cont...	CPU	Memor	Restart	Controlled B	Node	QoS	Age	Status
two-containers	default		N/A	N/A	0		worker01	BestEffort	40s	Running

Pod Context Menu Options:

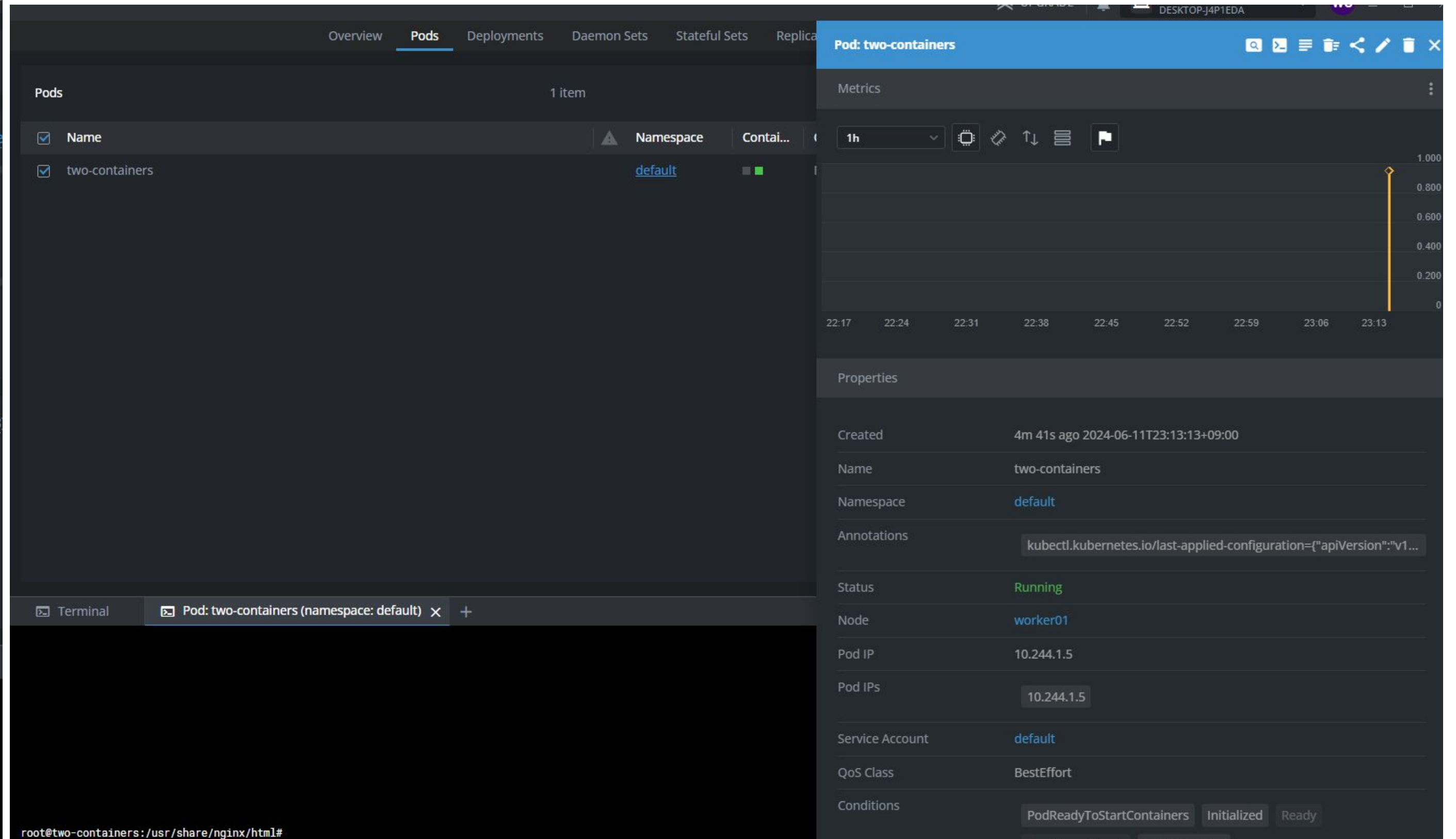
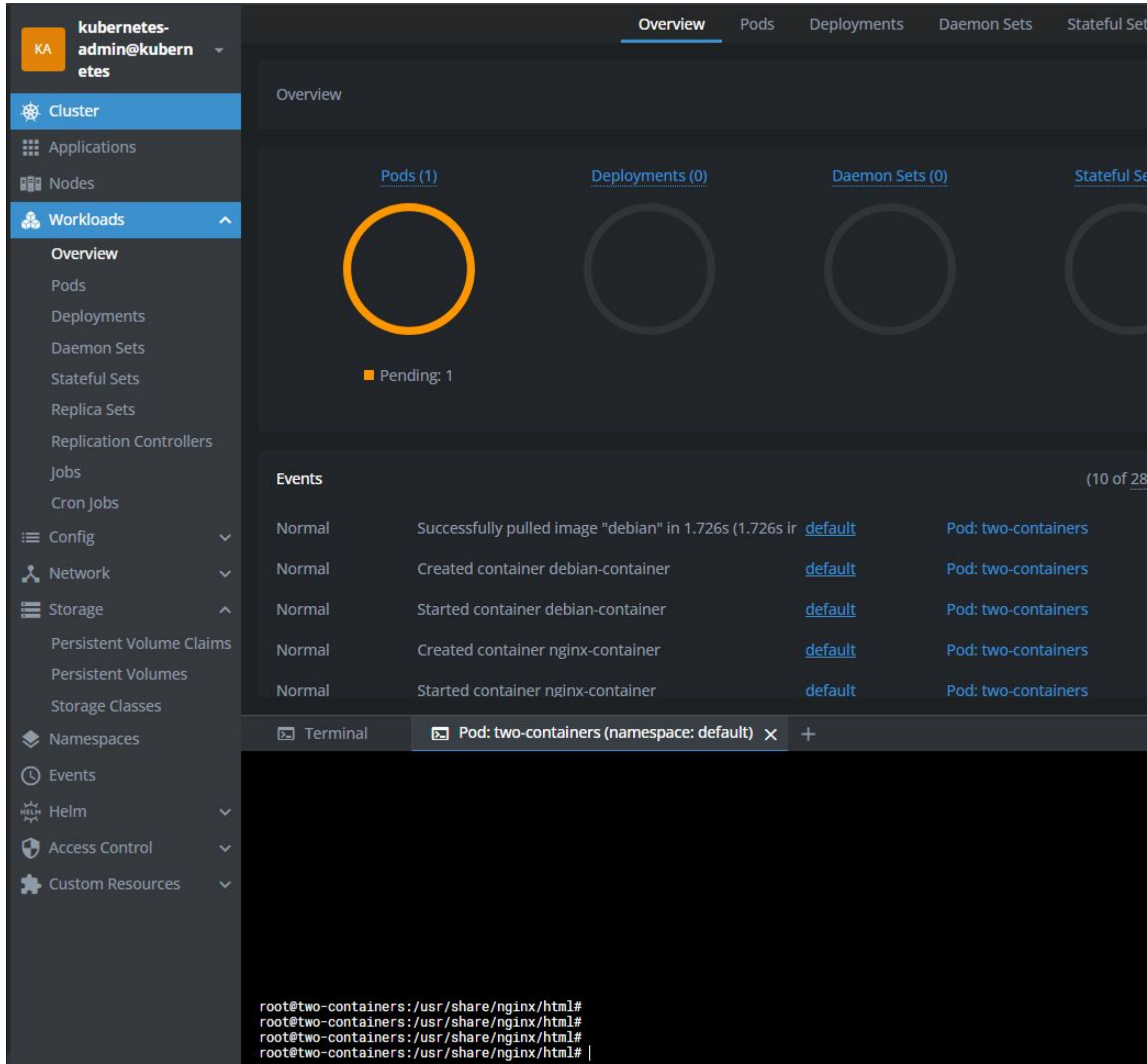
- Attach Pod
- Shell
- Evict
- Share link...
- Logs >
- Edit
- Delete

Terminal Output:

```
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/04/18 14:11:07 [notice] 1#1: using the "epoll" event method
2025/04/18 14:11:07 [notice] 1#1: nginx/1.27.5
2025/04/18 14:11:07 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2025/04/18 14:11:07 [notice] 1#1: OS: Linux 6.8.0-58-generic
2025/04/18 14:11:07 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
```


LENS 설치

- LENS 설정 및 확인



crictl

[참고] 컨테이너 런타임 관리도구

- `ctr -n k8s.io container list`
- `ctr -n k8s.io image list`
- `crictl`
- `crictl pods`
- `crictl images`

`crictl, ctr`은 **현재 노드의** containerd 상태를 확인하는 명령임

Replicas

Replicas

- k apply -f repltest.yml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # 케이스에 따라 레플리카를 수정한다.
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: mynginx
          image: nginx:latest
```

- k get rs

selector와 template.labels가 반드시 일치해야 함

이것이 ReplicaSet이 "어떤 Pod를 관리할지"를 결정하는 기준

Replicas

You can then get the current ReplicaSets deployed:

```
kubectl get rs
```

And see the frontend one you created:

NAME	DESIRED	CURRENT	READY	AGE
frontend	3	3	3	6s

You can also check on the state of the ReplicaSet:

```
kubectl describe rs/frontend
```

```
root@kopo:~/project/ex03-replica# k get pods --show-labels
NAME                                READY STATUS RESTARTS AGE LABELS
frontend-jb9kl                      1/1   Running 0       54s tier=frontend
frontend-ksbz8                      1/1   Running 0       54s tier=frontend
frontend-vcpsm                      1/1   Running 0       54s tier=frontend
kubernetes-bootcamp-6f6656d949-sdhqp 1/1   Running 0      10h app=kubernetes-bootcamp,pod-template-hash=6f6656d949
root@kopo:~/project/ex03-replica# k label pod/frontend-jb9kl tier=
pod/frontend-jb9kl labeled
root@kopo:~/project/ex03-replica# k get pods --show-labels
NAME                                READY STATUS RESTARTS AGE LABELS
frontend-4jbf6                      1/1   Running 0        4s tier=frontend
frontend-jb9kl                      1/1   Running 0      3m6s <none>
frontend-ksbz8                      1/1   Running 0      3m6s tier=frontend
frontend-vcpsm                      1/1   Running 0      3m6s tier=frontend
kubernetes-bootcamp-6f6656d949-sdhqp 1/1   Running 0      10h app=kubernetes-bootcamp,pod-template-hash=6f6656d949
root@kopo:~/project/ex03-replica# k label pod/frontend-jb9kl tier=frontend
pod/frontend-jb9kl labeled
root@kopo:~/project/ex03-replica# k get pods --show-labels
NAME                                READY STATUS RESTARTS AGE LABELS
frontend-jb9kl                      1/1   Running 0      3m39s tier=frontend
frontend-ksbz8                      1/1   Running 0      3m39s tier=frontend
frontend-vcpsm                      1/1   Running 0      3m39s tier=frontend
kubernetes-bootcamp-6f6656d949-sdhqp 1/1   Running 0      10h app=kubernetes-bootcamp,pod-template-hash=6f6656d949
root@kopo:~/project/ex03-replica# k scale --replicas=6 -f repltest.yml
replicaset.apps/frontend scaled
root@kopo:~/project/ex03-replica# k get pods --show-labels
NAME                                READY STATUS RESTARTS AGE LABELS
frontend-8vwl6                      1/1   Running 0        5s tier=frontend
frontend-jb9kl                      1/1   Running 0      4m20s tier=frontend
frontend-ksbz8                      1/1   Running 0      4m20s tier=frontend
frontend-lzflt                      1/1   Running 0        5s tier=frontend
frontend-vbthb                      1/1   Running 0        5s tier=frontend
frontend-vcpsm                      1/1   Running 0      4m20s tier=frontend
kubernetes-bootcamp-6f6656d949-sdhqp 1/1   Running 0      10h app=kubernetes-bootcamp,pod-template-hash=6f6656d949
```

Delete label of one pod

Set the label

Add replica number

Replicas

k describe rs/<replicas name>

```
root@master:~/kopoK8sPractice/Practice/ex03-replica# k describe rs/frontend
Name:          frontend
Namespace:     default
Selector:      tier=frontend
Labels:        app=guestbook
               tier=afrontend
Annotations:   <none>
Replicas:      3 current / 3 desired
Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  tier=frontend
  Containers:
    php-redis:
      Image:      gcr.io/google_samples/gb-frontend:v3
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>
```


Deployment

Deployment

- k apply -f deploytest.yml
- kubectl get deployments
kubectl rollout status deployment nginx-deployment
kubectl get rs
kubectl get pods --show-labels

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Deployment

Deployment update

- `kubectl edit deployments.apps nginx-deployment`

In editor window, modify nginx version
1.14.2 -> 1.16.1

- `kubectl rollout status deployment nginx-deployment`

```
root@master:~/kopoK8sPractice/Practice/ex04-deployment# k rollout status deployment nginx-deployment
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...

Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "nginx-deployment" successfully rolled out
```

- `kubectl describe deployments`

Deployment

Deployment roll-back

이전 실습 초기화 : `kubectl delete -f deploytest.yml`

- `k apply -f deploytest.yml`

`k rollout history deployment nginx-deployment`
`k rollout history deployment nginx-deployment --revision=1`

- 버전 업 배포(1.14.2 -> 1.16.1) : `k apply -f deploytest2.yml`
- `kubectl rollout history deployment.v1.apps/nginx-deployment --revision=2`
- `kubectl rollout undo deployment.v1.apps/nginx-deployment --to-revision=1`

Or
`kubectl rollout undo deployment.v1.apps/nginx-deployment`

- | REVISION | CHANGE-CAUSE |
|----------|------------------------------------|
| 2 | Initial deployment with Nginx 1.61 |
| 3 | <none> |

 ment

Deployment

Scaling

Scaling a Deployment

You can scale a Deployment by using the following command:

```
kubectl scale deployment.v1.apps/nginx-deployment --replicas=10
```

The output is similar to this:

```
deployment.apps/nginx-deployment scaled
```

Assuming [horizontal Pod autoscaling](#) is enabled in your cluster, you can setup an autoscaler for your Deployment and choose the minimum and maximum number of Pods you want to run based on the CPU utilization of your existing Pods.

```
kubectl autoscale deployment.v1.apps/nginx-deployment --min=10 --max=15 --cpu-percent=80
```

The output is similar to this:

```
deployment.apps/nginx-deployment scaled
```


deployment

Rolling update

Rolling Update Deployment [↗](#)

The Deployment updates Pods in a rolling update fashion when `.spec.strategy.type==RollingUpdate`. You can specify `maxUnavailable` and `maxSurge` to control the rolling update process.

Max Unavailable

`.spec.strategy.rollingUpdate.maxUnavailable` is an optional field that specifies the maximum number of Pods that can be unavailable during the update process. The value can be an absolute number (for example, 5) or a percentage of desired Pods (for example, 10%). The absolute number is calculated from percentage by rounding down. The value cannot be 0 if `.spec.strategy.rollingUpdate.maxSurge` is 0. The default value is 25%.

For example, when this value is set to 30%, the old ReplicaSet can be scaled down to 70% of desired Pods immediately when the rolling update starts. Once new Pods are ready, old ReplicaSet can be scaled down further, followed by scaling up the new ReplicaSet, ensuring that the total number of Pods available at all times during the update is at least 70% of the desired Pods.

Max Surge

`.spec.strategy.rollingUpdate.maxSurge` is an optional field that specifies the maximum number of Pods that can be created over the desired number of Pods. The value can be an absolute number (for example, 5) or a percentage of desired Pods (for example, 10%). The value cannot be 0 if `maxUnavailable` is 0. The absolute number is calculated from the percentage by rounding up. The default value is 25%.

For example, when this value is set to 30%, the new ReplicaSet can be scaled up immediately when the rolling update starts, such that the total number of old and new Pods does not exceed 130% of desired Pods. Once old Pods have been killed, the new ReplicaSet can be scaled up further, ensuring that the total number of Pods running at any time during the update is at most 130% of desired Pods.

✅ Rolling Update란?

- Deployment 리소스가 사용하는 기본 업데이트 전략
- 기존 버전의 Pod를 조금씩 줄이면서, 새로운 버전의 Pod를 조금씩 늘려서 배포
- 클러스터 자원과 서비스 가용성을 최대한 유지하면서 버전 업그레이드 가능

⚙️ 동작 방식

```
yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

필드	설명
<code>maxUnavailable</code>	업데이트 중 동시에 중지될 수 있는 Pod 수 (기본값: 25%)
<code>maxSurge</code>	기존 replica 수보다 더 많이 동시에 생성할 수 있는 새 Pod 수 (기본값: 25%)

예: replicas: 4, `maxUnavailable: 1`, `maxSurge: 1`
→ 동시에 최대 5개까지 Pod 존재 가능, 최소 3개는 항상 살아있도록 유지

There are many strategies like
‘recreate’, ‘deadline seconds’ and so on.

참고...

✅ 세 가지 배포 전략 비교표

전략	개요	장점	단점	적용 예
Rolling Update	기존 Pod을 순차적으로 새 버전으로 교체	무중단, 자동화 쉬움	구버전과 신버전이 혼재 (문제 발생 시 감지 어려움)	일반 서비스 업데이트
Blue-Green	두 개의 버전(Blue/Green)을 완전히 분리하여 배포 후, 트래픽 전환	간단한 롤백, 버전 간 격리	리소스 2배 필요, 배포 자동화 필요	민감한 서비스 업데이트
Canary	소수의 사용자에게만 새 버전을 점진적으로 배포하고 반응을 보고 확대	안정성 높음, 실시간 실험 가능	트래픽 라우팅 및 모니터링 복잡	대규모 서비스 실험/검증

Service

Service

cluster ip (internal network)

- k apply -f clusterip.yml

Creating a Service

So we have pods running nginx in a flat, cluster wide, address space. In theory, you could talk to these pods directly, **but what happens when a node dies?** The pods die with it, and the **Deployment will create new ones, with different IPs.**

This is the problem a Service solves.

A Kubernetes Service is an abstraction which defines a logical set of Pods running somewhere in your cluster, that all provide the same functionality.

When created, each **Service is assigned a unique IP address (also called clusterIP).**

This address is tied to the lifespan of the Service,

and will not change while the Service is alive.

Pods can be configured to talk to the Service,

and know that communication to the Service will be automatically load-balanced out to some pod that is a member of the Service.

Client → Service (my-nginx:80) → Pod (my-nginx x 2) → nginx 컨테이너

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels:
      run: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        run: my-nginx
    spec:
      containers:
        - name: my-nginx
          image: nginx
          ports:
            - containerPort: 80

---
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  ports:
    - port: 80
      protocol: TCP
  selector:
    run: my-nginx
```


Service

cluster ip (internal network)

- k get all
- kubectl get pods -l run=my-nginx -o wide
- kubectl get pods -l run=my-nginx -o yaml | grep podIP
- kubectl get svc my-nginx
- kubectl describe svc my-nginx
- kubectl get ep my-nginx
- kubectl exec my-nginx-5dc4865748-rhfq8 -- printenv | grep SERVICE

```
root@kopo:~/project/ex04-deployment# k get all
NAME                                READY STATUS RESTARTS AGE
pod/kubernetes-bootcamp-6f6656d949-sdhqp  1/1   Running  1    23h
pod/my-nginx-5dc4865748-rhfq8             1/1   Running  0     36s
pod/my-nginx-5dc4865748-z7qkt             1/1   Running  0     36s

NAME                                TYPE      CLUSTER-IP   EXTERNAL-IP  PORT(S)  AGE
service/kubernetes  ClusterIP  10.96.0.1    <none>       443/TCP  2d19h
service/my-nginx    ClusterIP  10.110.148.126 <none>      80/TCP   36s

NAME                                READY UP-TO-DATE AVAILABLE AGE
deployment.apps/kubernetes-bootcamp  1/1   1           1    23h
deployment.apps/my-nginx              2/2   2           2    36s

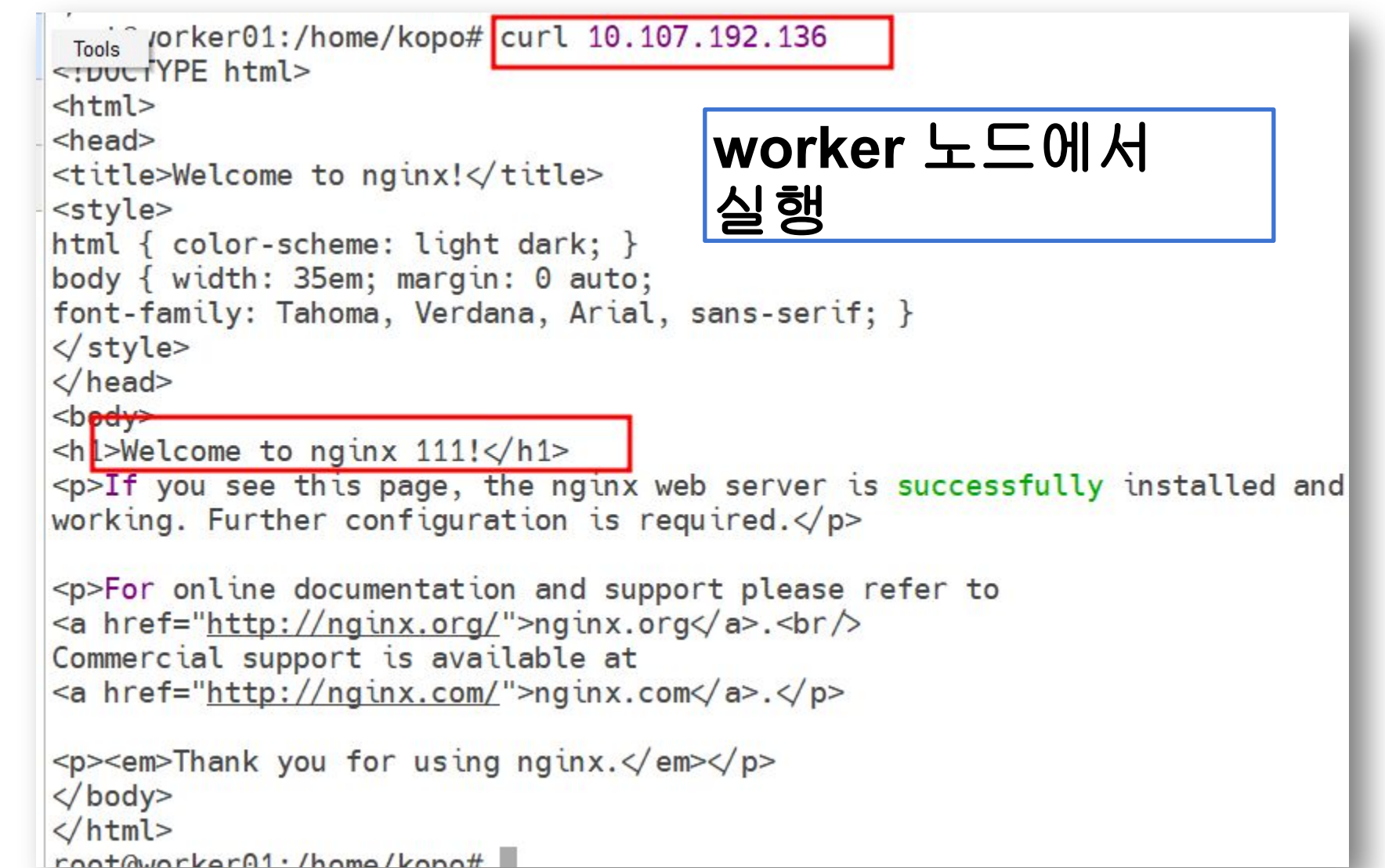
NAME                                DESIRED CURRENT READY AGE
replicaset.apps/kubernetes-bootcamp-6f6656d949  1      1      1    23h
replicaset.apps/my-nginx-5dc4865748             2      2      2    36s
root@kopo:~/project/ex04-deployment# kubectl get pods -l run=my-nginx -o wide
NAME                                READY STATUS RESTARTS AGE IP      NODE    NOMINATED NODE
READINESS GATES
my-nginx-5dc4865748-rhfq8  1/1   Running  0      60s  10.244.1.28  worker01  <none>    <none>
my-nginx-5dc4865748-z7qkt  1/1   Running  0      60s  10.244.1.27  worker01  <none>    <none>
root@kopo:~/project/ex04-deployment# kubectl get pods -l run=my-nginx -o yaml | grep podIP
  f:podIP: {}
  f:podIPs:
    podIP: 10.244.1.28
  podIPs:
    f:podIP: {}
    f:podIPs:
      podIP: 10.244.1.27
  podIPs:
root@kopo:~/project/ex04-deployment# kubectl get svc my-nginx
NAME    TYPE      CLUSTER-IP   EXTERNAL-IP  PORT(S)  AGE
my-nginx  ClusterIP  10.110.148.126 <none>       80/TCP   104s
root@kopo:~/project/ex04-deployment# kubectl describe svc my-nginx
Name:      my-nginx
Namespace: default
Labels:    run=my-nginx
Annotations: Selector: run=my-nginx
Type:      ClusterIP
IP:        10.110.148.126
Port:      <unset> 80/TCP
TargetPort: 80/TCP
Endpoints:  10.244.1.27:80,10.244.1.28:80
Session Affinity: None
Events:     <none>
root@kopo:~/project/ex04-deployment# kubectl get ep my-nginx
NAME    ENDPOINTS                                AGE
my-nginx  10.244.1.27:80,10.244.1.28:80  2m3s

root@kopo:~/project/ex04-deployment# kubectl exec my-nginx-5dc4865748-rhfq8 -- printenv | grep
SERVICE
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_HOST=10.96.0.1
MY_NGINX_SERVICE_PORT=80
KUBERNETES_SERVICE_PORT=443
```

클러스터 IP로 접속 확인

Pod <-> Local File Copy

- Pod의 Nginx 에서 서비스하는 index.html 파일을 로컬로 복사
k cp default/my-nginx-646554d7fd-gqsfh:usr/share/nginx/html/index.html ./index.html
- index.html을 편집하여 Pod간 index.html 파일 내용을 다르게 구분해 둔다.
- 편집한 index.html 파일을 Pod로 복사
k cp ./index.html default/my-nginx-646554d7fd-gqsfh:usr/share/nginx/html/index.html
- 해당 Pod로 진입하여 변경된 파일 확인
k exec -it my-nginx-646554d7fd-gqsfh -- /bin/bash
- 접속 확인



The screenshot shows a terminal window on a worker node. At the top, the prompt is 'worker01:/home/kopo#'. A red box highlights the command 'curl 10.107.192.136'. The output is an HTML page. A blue box on the right contains the text 'worker 노드에서 실행'. Another red box highlights the first line of the HTML body: '<h1>Welcome to nginx 111!</h1>'. The rest of the HTML content is visible below.

```
worker01:/home/kopo# curl 10.107.192.136
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx 111!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

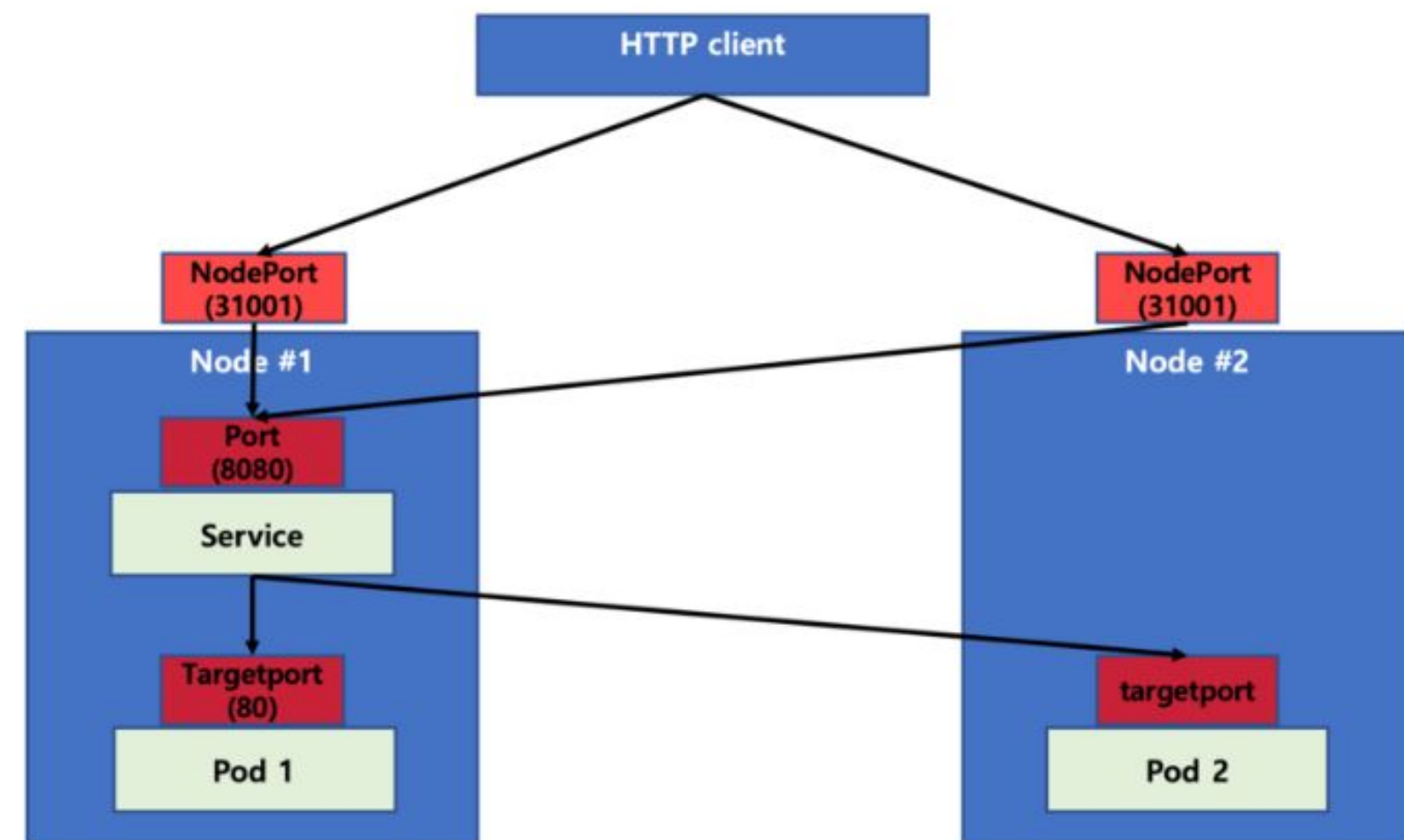
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
worker01:/home/kopo#
```


Service

Nodeport (expose network)

상황	설명
🔧 개발/테스트 환경에서 외부 브라우저로 직접 접속하고 싶을 때	예: curl http://<node-ip>:30080 또는 브라우저에서 Nginx 접속
🌐 클라우드가 아닌 Bare Metal 환경에서 외부 노출이 필요할 때	LoadBalancer가 없기 때문에 NodePort가 유일한 대안
🔧 Ingress를 구성하기 전에 임시로 노출해야 할 때	프론트엔드/백엔드 서비스 검증용 등

```
spec:
  ports:
  - nodePort: 31001
    port: 8080
    targetPort: 80
    protocol: TCP
  selector:
    app: helloworld
  type: NodePort
```



```
type: NodePort
ports:
- name: http
  port: 80
  protocol: TCP
  targetPort: 8080
  nodePort: 30036
```

아래 그림과 같은 구조가 된다.

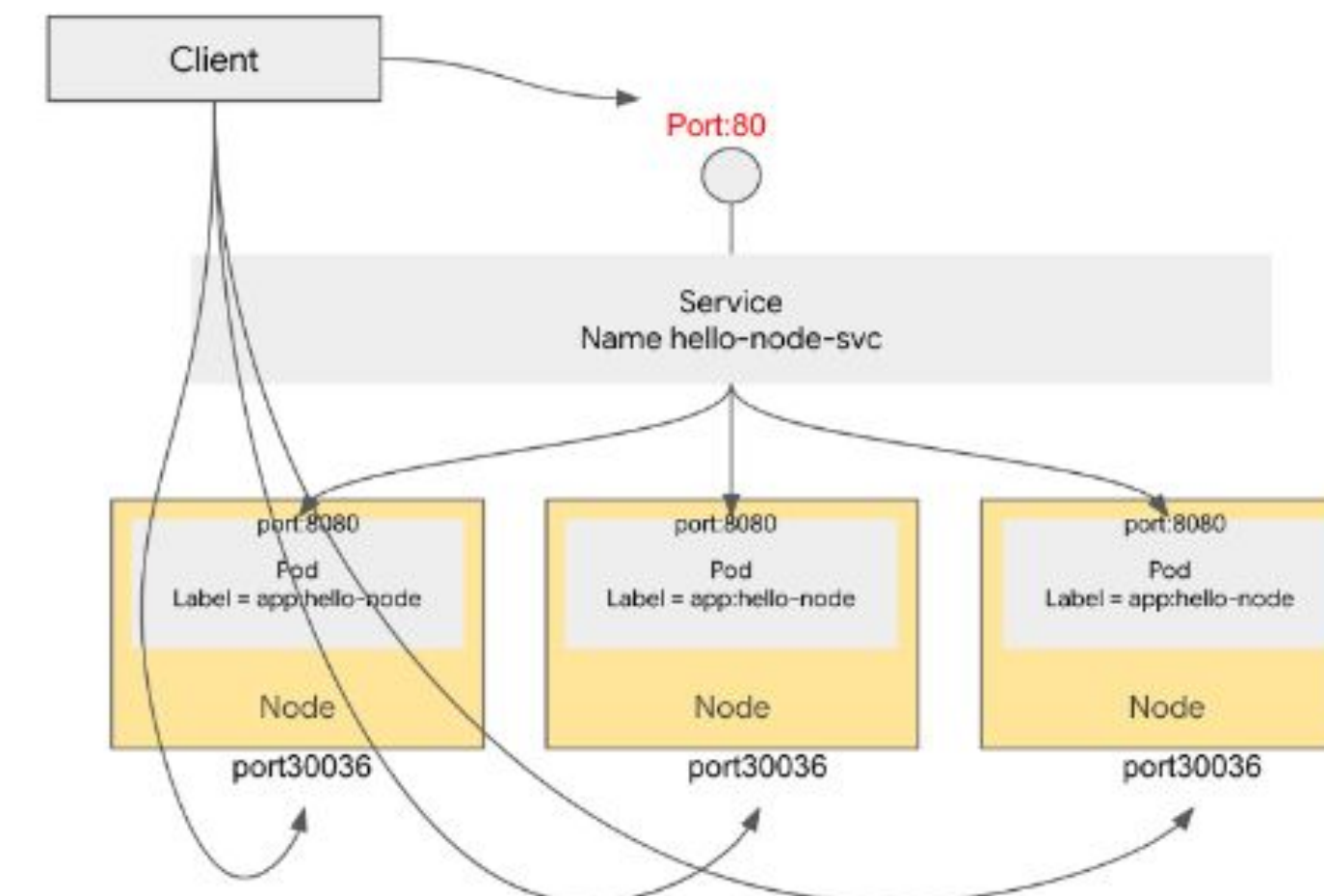


그림 출처:
<https://bcho.tistory.com/tag/nodeport>

Service

Nodeport (expose network)

- K apply -f nodeport.yml

curl http://192.168.56.61:30101

Reference: <https://bcho.tistory.com/tag/nodeport>

Name	my-nginx	
Namespace	default	
Labels	run=my-nginx	
Annotations	kubectl.kubernetes.io/last-applie	
Selector	run=my-nginx	
Type	NodePort	
Session Affinity	None	
Connection		
Cluster IP	10.109.174.238	
Cluster IPs	10.109.174.238	
IP families	IPv4	
IP family policy	SingleStack	
Ports	80:30101/TCP	
Endpoint Slices		
Name	Endpoints	Ports
my-nginx-n5lms	2/2	80/TCP

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels:
      run: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        run: my-nginx
    spec:
      containers:
        - name: my-nginx
          image: nginx
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  type: NodePort
  ports:
    - protocol: TCP
      nodePort: 30101
      port: 80
      targetPort: 80
  selector:
    run: my-nginx
```

Service

at worker node

```
root@master:~/kopoK8sPractice/Practice/ex06-nodeport# k describe svc my-nginx
Name: my-nginx
Namespace: default
Labels: run=my-nginx
Annotations: <none>
Selector: run=my-nginx
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.108.110.100
IPs: 10.108.110.100
Port: <unset> 80/TCP
TargetPort: 80/TCP
NodePort: <unset> 30101/TCP
Endpoints: 10.244.1.65:80,10.244.1.64:80
Session Affinity: None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events: <none>
```

curl
10.108.110.100

curl
localhost:30101

curl 10.244.1.64:80

curl 10.244.1.65:80

정리

타입	접근 가능 위치	기본 포트	사용 목적
ClusterIP (기본값)	클러스터 내부에서만 접근 가능	내부 IP	마이크로서비스 간 통신, 백엔드 호출
NodePort	클러스터 외부에서 접근 가능	노드의 IP + 고정 포트 (30000~32767)	브라우저/사용자 직접 접근, 외부 테스트
LoadBalancer	클라우드 환경에서 퍼블릭 IP 제공	외부 IP	운영 서비스에 외부 트래픽 수용

LoadBalancer

개요

- Pod들과 통신하기 위한 **Service** 리소스 생성

```
yaml  
  
spec:  
  type: LoadBalancer
```

- 외부에 노출되는 퍼블릭 IP를 제공하는 서비스 타입
- 클라우드 환경(GKE, EKS, AKS 등)에서 외부 IP를 자동으로 할당
- Bare Metal 환경에서는 MetalLB 같은 외부 LoadBalancer가 필요

```
[외부 사용자] -(80/TCP)→ [LoadBalancer Service: my-nginx]  
|  
└─> Pod 1 (nginx:80)  
└─> Pod 2 (nginx:80)
```

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: my-nginx  
spec:  
  selector:  
    matchLabels:  
      run: my-nginx  
  replicas: 2  
  template:  
    metadata:  
      labels:  
        run: my-nginx  
    spec:  
      containers:  
        - name: my-nginx  
          image: nginx  
          ports:  
            - containerPort: 80  
  
---  
apiVersion: v1  
kind: Service  
metadata:  
  name: my-nginx  
  labels:  
    run: my-nginx  
spec:  
  type: LoadBalancer  
  ports:  
    - protocol: TCP  
      port: 80  
      targetPort: 80  
  selector:  
    run: my-nginx
```

개요

✅ 실행 시 기대 결과

1. 클라우드 환경이면 `kubectl get svc` 시 외부 IP가 할당됨

```
bash
```

[Copy](#)[Edit](#)

```
kubectl get svc my-nginx
```

```
pgsql
```

[Copy](#)[Edit](#)

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-nginx	LoadBalancer	10.96.0.123	34.80.12.45	80:32456/TCP	2m

2. 브라우저나 curl로 접속:

```
nginx
```

[Copy](#)[Edit](#)

```
curl http://<EXTERNAL-IP>
```


MetalLB

로컬 / Bare Metal 환경에 클라우드처럼 외부 IP를 할당할 수 있게 해주는 네트워크 로드밸런서

- 클라우드 환경(GKE, EKS 등)은 LoadBalancer 타입 사용 시 외부 IP 자동 할당됨
- Bare Metal 환경에서는 **LoadBalancer** Service에 외부 IP를 할당·광고해 줄 구현체가 기본 제공되지 않음

```
kubectl apply -f metallb-native.yaml
```

```
kubectl get pods -n metallb-system
```

```
kubectl apply -f metallb-config.yaml
```

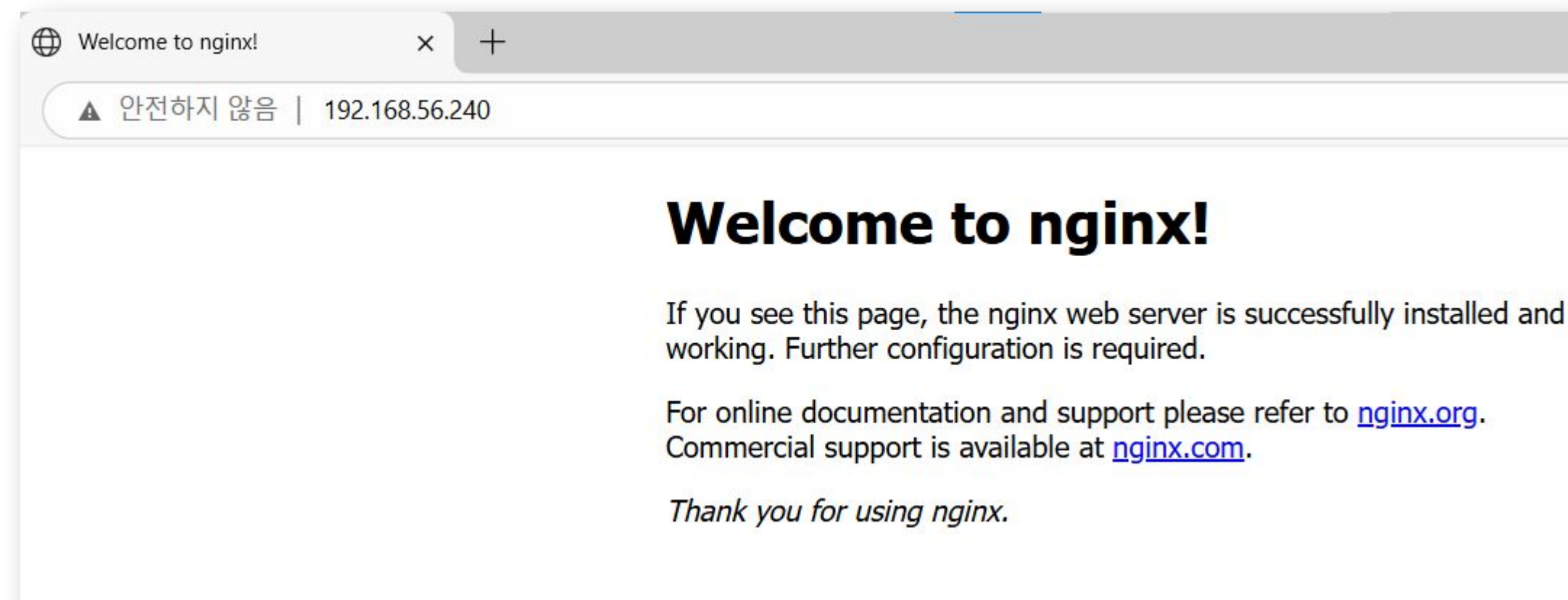
```
kubectl get ipaddresspool,ipaddress -n metallb-system
```

결과 확인

k apply -f loadbalancer.yml

```
# kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	2d1h
my-nginx	LoadBalancer	10.111.185.146	192.168.56.240	80:31298/TCP	27m



Connection	
Cluster IP	10.102.43.220
Cluster IPs	10.102.43.220
IP families	IPv4
IP family policy	SingleStack
External IPs	192.168.56.240

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels:
      run: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        run: my-nginx
    spec:
      containers:
        - name: my-nginx
          image: nginx
          ports:
            - containerPort: 80
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  selector:
    run: my-nginx
```

Ingress

클러스터 외부의 HTTP/HTTPS 요청을 클러스터 내부의 서비스(Service)로 라우팅 해주는 리소스

여러 도메인과 경로를 하나의 엔드포인트로 받아 처리할 수 있도록 해주는 **트래픽 라우터**

[사용자 브라우저]

| http://example.com



[Ingress Controller] ← Ingress 규칙



[Service] → [Pod]

Ingress

<https://kubernetes.github.io/ingress-nginx/deploy/#bare-metal>

- wget
https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.15.1/deploy/static/provider/baremetal/deploy.yaml
- k apply -f deploy.yaml

kubectl get pods -n ingress-nginx

kubectl get svc -n ingress-nginx

kubectl get ingressclass

kubectl get all -n ingress-nginx

ingress test

basic.yml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: test-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - http:
        paths:
          - path: /testpath
            pathType: Prefix
        backend:
          service:
            name: test
```

```
root@kopo:~/project/ex08-ingress# k get ingress
```

```
root@master:/home/kopo/kopoK8sPractice/Practice/ex08-ingress# k get ingress
NAME          CLASS      HOSTS      ADDRESS      PORTS      AGE
test-ingress  <none>    *          *            80         61s
```

Home

default

Aa

Search Ingresses...

1 item

KUBERNETES CLUSTERS

Local Kubeconfigs

kubernetes-admin@kubernetes

Overview

Applications

Nodes

Workloads

Overview

Pods

Deployments

Daemon Sets

Stateful Sets

Replica Sets

Replication Controllers

Jobs

Cron Jobs

Config

Network

Services

Endpoint Slices

Endpoints

Ingresses

Ingress Classes

Network Policies

Port Forwarding

Storage

Namespaces

Events

Name

Namespace

LoadBalancers

test-ingress

default

Terminal

Pod two-containers

Pod: frontend-2dpg6

Pod: frontend-cp9d6

Resource not found

Ingress: test-ingress

1h

Displaying metrics from Kubernetes Metrics Server

Properties

Created

100s ago (2026년 8월 16일 오후 10시 0분 51초 GMT+9)

Name

test-ingress

Namespace

default

Annotations

2 Annotations

Ports

80

Rules

Path

Link

Backends

/testpath

http://*/testpath

test:80

Ingress

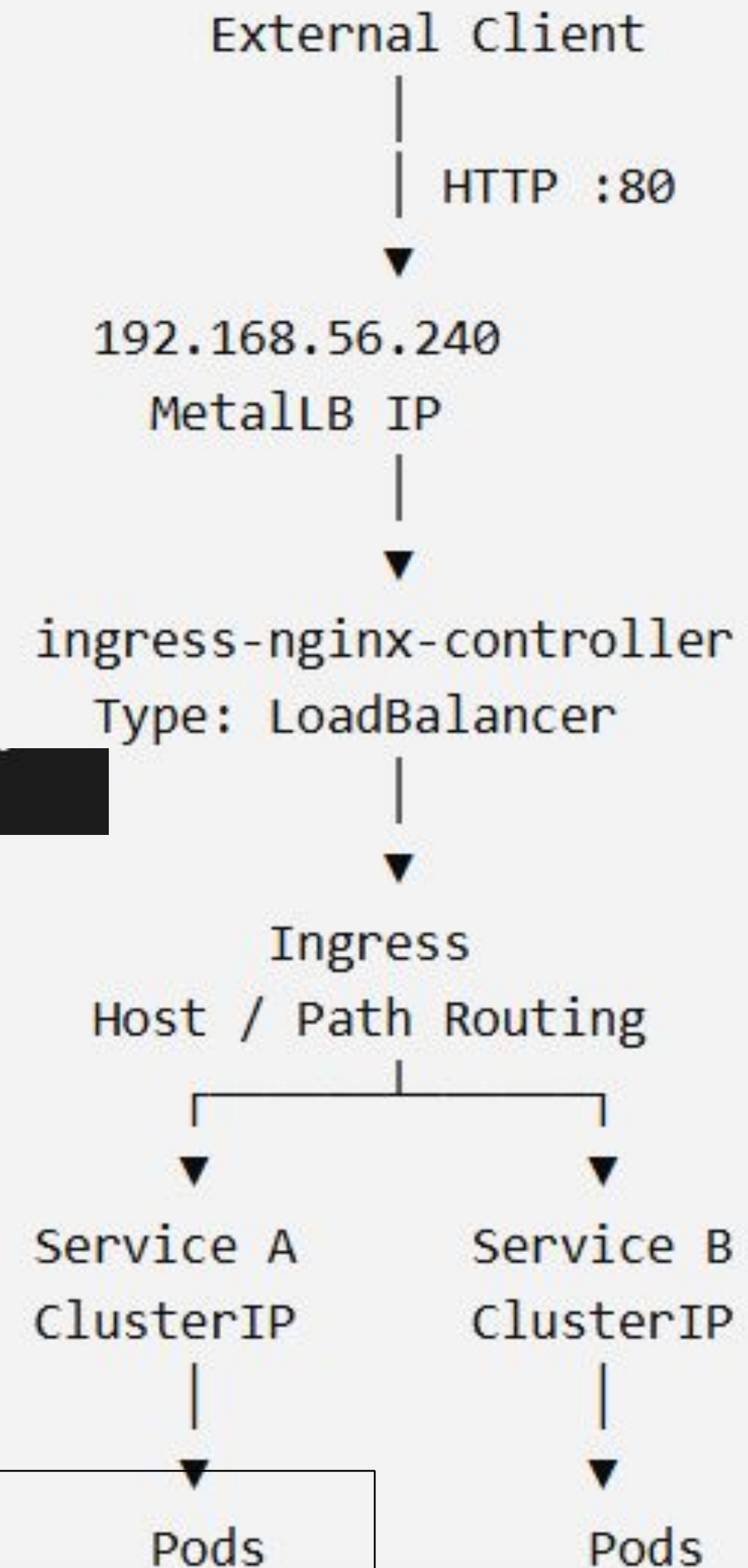
Ingress Controller 앞에 **LoadBalancer** Service를 두어 외부 IP를 확보하는 구성

```
kubectl patch svc ingress-nginx-controller \
-n ingress-nginx \
-p '{"spec":{"type":"LoadBalancer"}}'
```

확인>

```
kubectl get svc ingress-nginx-controller -n ingress-nginx
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
ingress-nginx-controller	LoadBalancer	10.105.44.81	192.168.56.240	80:32465/TCP,443:30204/TCP	22m



MetalLB = "외부에서 들어올 IP를 제공"

Ingress = "들어온 HTTP 요청을 어느 Service로 보낼지 결정"

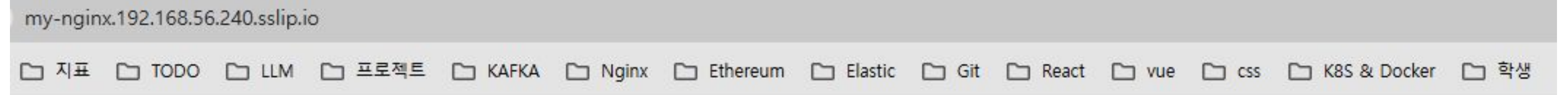
Ingress

Apply and test

- k apply -f ingress.yml

```
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
kubectl get ingressclass
kubectl get all -n ingress-nginx
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  type: LoadBalancer
  ports:
  - protocol: TCP
    # nodePort: 30101
    port: 80
    targetPort: 80
  selector:
    run: my-nginx
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

예시(하이퍼레저 프로젝트)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: "nginx"
  rules:
  - host: 192.168.56.61.sslip.io
    http:
      paths:
      - path: /()(.* )
        pathType: Prefix
        backend:
          service:
            name: explorer
            port:
              number: 8080
      - path: /api1(/|$)(.*)
        pathType: Prefix
        backend:
          service:
            name: rest-peer0-org1
            port:
              number: 3000
      - path: /api2(/|$)(.*)
        pathType: Prefix
        backend:
          service:
            name: rest-peer0-org2
            port:
              number: 3000
      - path: /()explorer(.*)
        pathType: Prefix
        backend:
          service:
            name: explorer
            port:
              number: 8080
```

sslip.io 서비스를 활용하여 로컬PC환경에서
도메인 rewrite 테스트를 진행한다.

일종의 정규표현식과 같은 형태로 URL을 Rewrite한다.

- 192.168.56.61은 Worker node의 IP
- Worker Node가 2대 이상일 경우 서비스 spec에 type을 NodePort로 명시 필요(확인중)

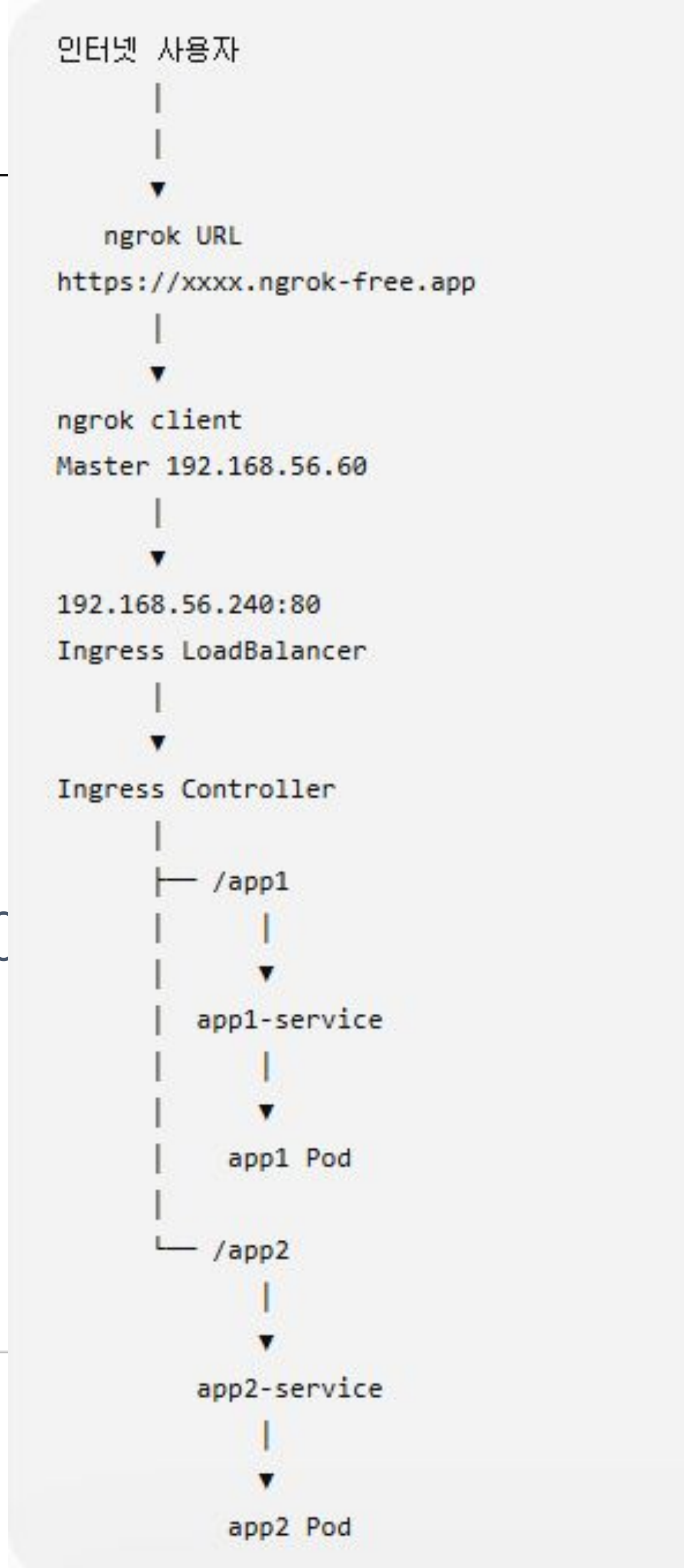
http://192.168.56.61 → explorer:8080
http://192.168.56.61/api1/query?~ (블록체인 조회) → rest-peer0-org1:3000/query?~
http://192.168.56.61/ap2/invoke (블록 생성) → rest-peer0-org2:3000/invoke
http://192.168.56.61/explorer → explorer:8080

<https://kubernetes.github.io/ingress-nginx/examples/rewrite/>

<https://github.com/kubernetes/ingress-nginx/issues/3122#issuecomment-576698063>

Ingress 경로 와 분팅

경로(Path)에 따른 Kubernetes Service 분기 처리 및 트래픽 제어
실습
아키텍처 구현 가이드



OBJECTIVE 02

실습

목표

01

단일 접속 포인트 구성

하나의 외부 주소로 전체 서비스에 접근 가능한 환경을 구축합니다.

02

ngrok 외부 노출

로컬 K8s 클러스터를 공인 도메인을 통해 외부로 안전하게 노출합니다.

03

경로 기반 라우팅

URL Path 조건에 따라 트래픽을 지능적으로 분산 처리합니다.

ROUTING RULES

/app1
→ app1-service

/app2
→ app2-service



Ingress Controller 기반

PREPARATION

실습 환경 구성

NODES CONFIGURATION

Master Node Control Plane	192.168.56.60
Worker01 Compute Node	192.168.56.61

 ENVIRONMENT

VirtualBox Host-Only Network 기반의
정적 IP 대역을 사용하여 클러스터를 구성합니다.

SERVICE ENTRY POINTS

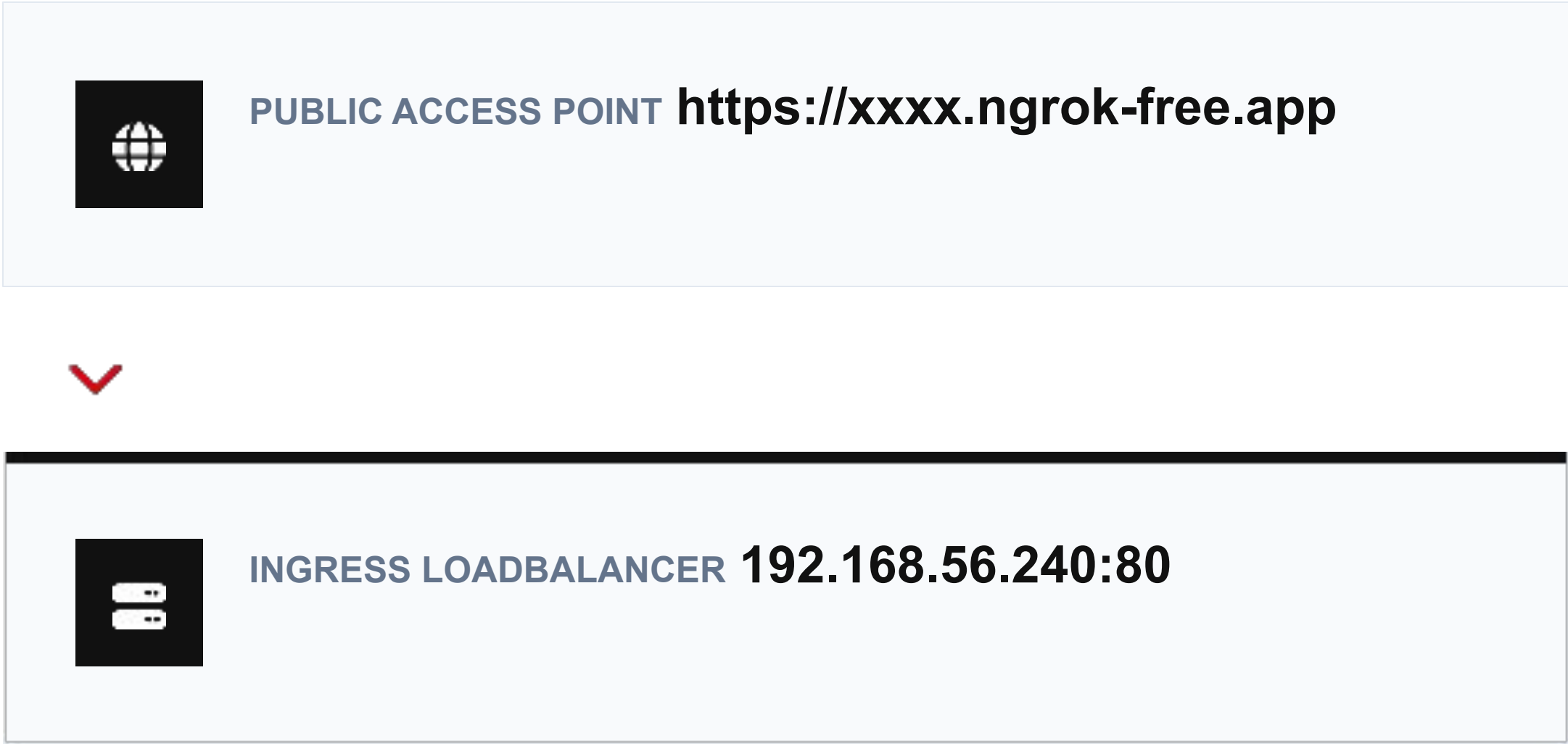
INGRESS LOADBALANCER 외부 요청 수신 (LB) 192.168.56.240

NGROK RUNTIME 터널링 실행 위치 192.168.56.60

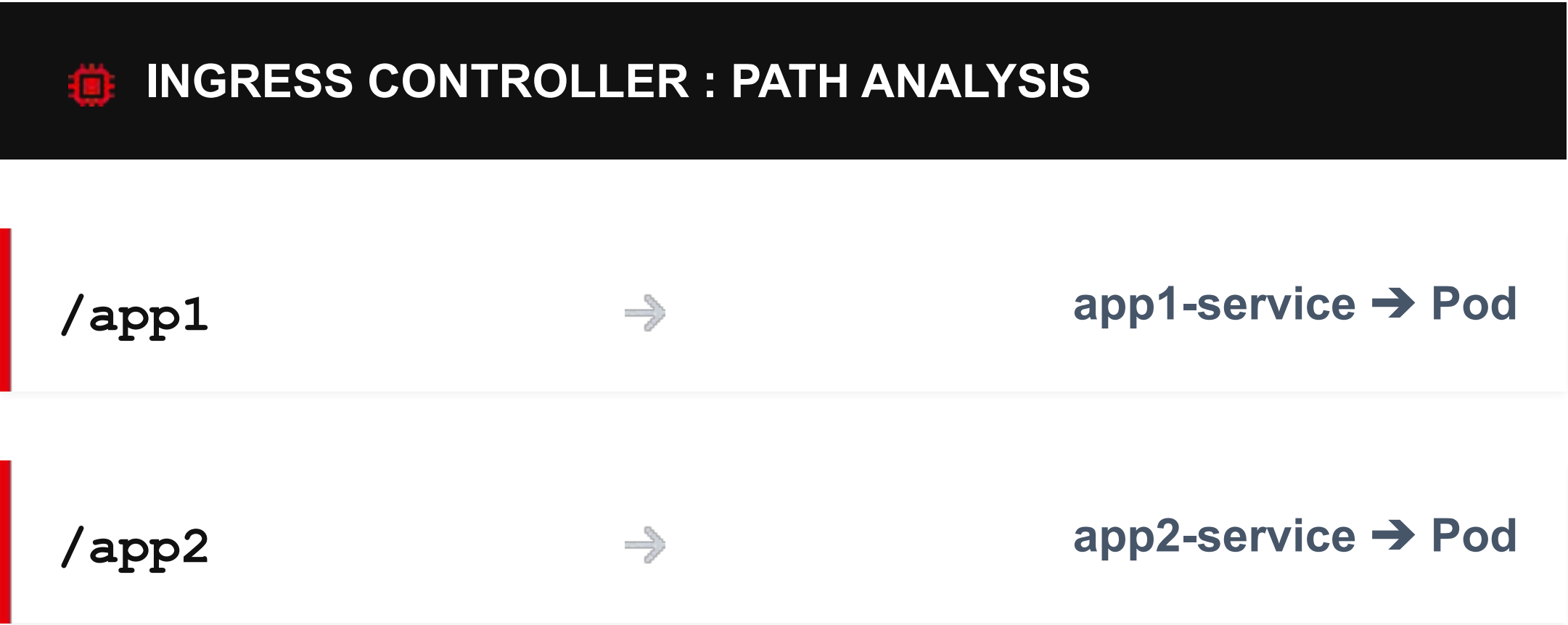
 주의사항 : ngrok 구동 위치와 트래픽 도달 IP(LB)는 기술적으로 분리되어 있습니다.

전체 아키텍처 흐름도

요청 흐름도 입 구조



내부 라우팅 구조



 L7 계층에서 HTTP 요청의 URI Path를 분석하여
사전에 정의된 서비스로 트래픽을 분기합니다.

CONFIGURATION

애플리케이션 구성

(Service)

ClusterIP

APP 2

ClusterIP

INGRESS ROUTING PATH /app1 → app1-service

- LoadBalancer 노출 없이 내부 네트워크에서만 통신
- Ingress를 통해서만 외부 접근 가능

INGRESS ROUTING PATH /app2 → app2-service

- LoadBalancer 노출 없이 내부 네트워크에서만 통신
- Ingress를 통해서만 외부 접근 가능

NETWORKING LAYER

Ingress의

역할 Ingress Controller

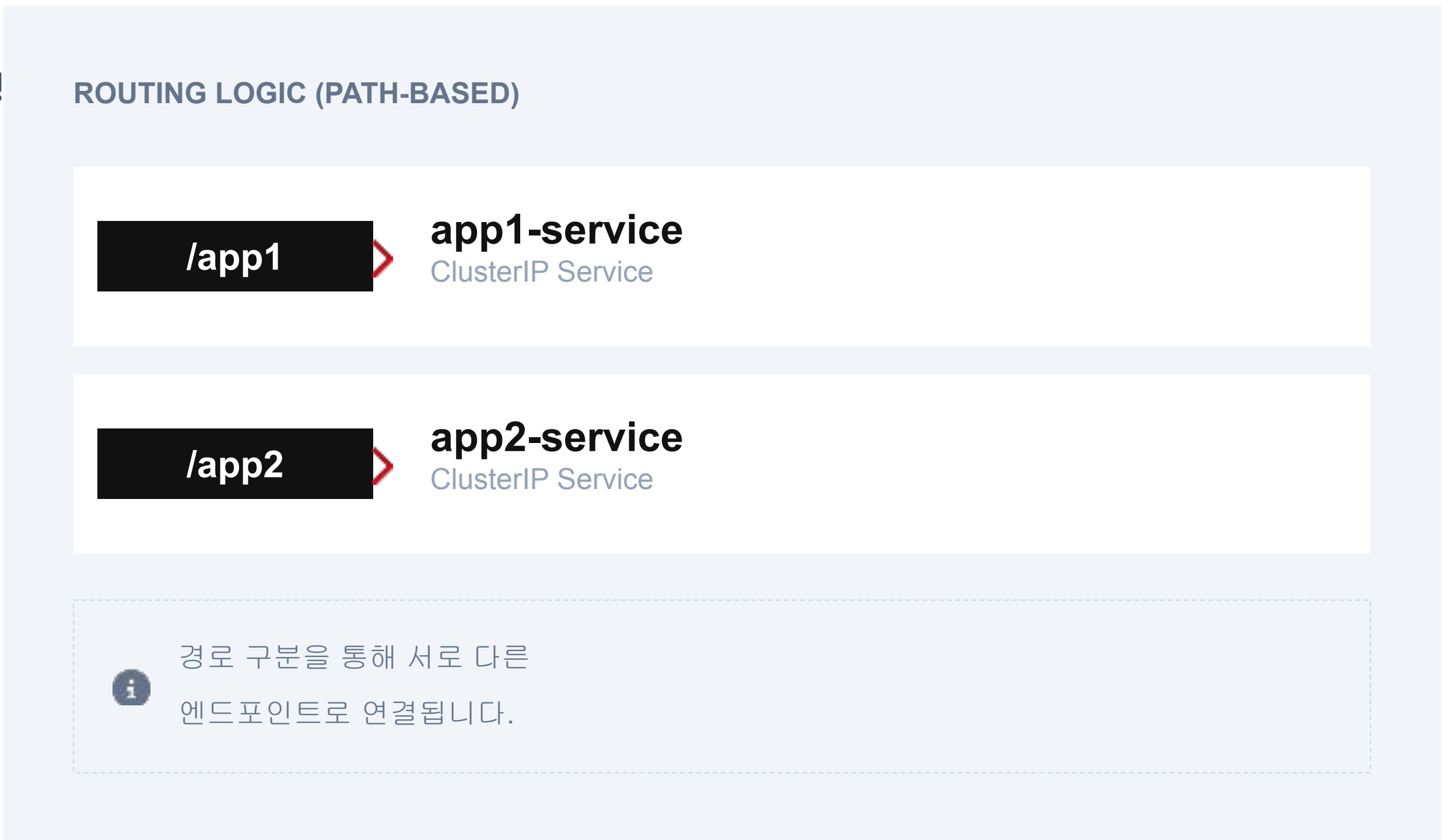
URL 경로(Path)를 분석하여 적절한 서비스로 트래픽을 유도하는 지능형 라우터입니다.

L7 로드밸런싱

HTTP/HTTPS 헤더와 URL을 기반으로 정밀한 트래픽 제어가 가능합니다.



하나의 LoadBalancer IP로 여러 서비스를 동시 호스팅



Ingress YAML 핵심

RULE 01

경로 (Path) 규칙

```
path: /app1
pathType: Prefix
```

■ 하위 경로 매칭 규칙 정의

/app1로 시작하는 모든 하위 URL을 포함합니다. 예: /app1/test, /app1/api/v1

RULE 02

백엔드 라우팅 (Backend)

```
service:
  name: app1-service
  port:
    number: 80
```

■ 서비스 및 포트 매핑

K8s Service 이름과 포트를 매핑하여 트래픽을 전달합니다.

■ 추상화된 유연한 구조

IP 대신 Service 이름을 사용하여 라우팅을 유지합니다.

CONFIGURATION

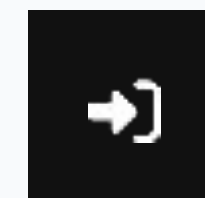
rewrite-target

어노테이션

```
nginx.ingress.kubernetes.io/  
rewrite-target: /
```

Ingress 컨트롤러가 요청을 가공하여 백엔드 서비스에 전달할 때 **경로를 재작성** 하기 위한 필수 어노테이션입니다.

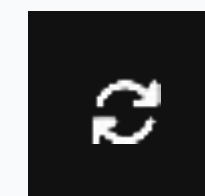
■ Rewrite (경로 재작성) 기능



사용자 요청 처리

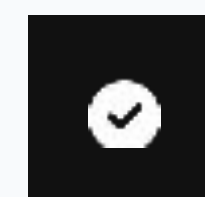
외부 사용자가 특정 경로(Path)로 서비스를 호출할 때 발생하는 매핑 이슈를 해결합니다.

```
GET http://도메인/app1
```



경로 변환 프로세스

Ingress가 요청된 /app1 경로를 실제 서비스의 루트인 / 경로로 변환하여 전달합니다.



기본 페이지 출력

웹 서버의 콘텐츠가 루트 경로에 존재할 경우, 정상적인 메인 페이지 출력을 위한 필수 설정입니다.

NETWORK VERIFICATION

ngrok 연결 및 외부 접속 테스트

STEP 01

내부 K8s 통신 테스트

```
# Ingress IP를 통한 내부 호출 확인
curl http://192.168.56.240/app1
curl http://192.168.56.240/app2
```

STEP 02

ngrok 터널링 오픈

```
# Ingress IP를 공인 도메인으로 매핑
ngrok http 192.168.56.240:80
```

STATUS

ENDPOINT

<https://abcd1234.ngrok-free.app>

VERIFICATION

외부 접근 확인

✅ /app1 및 /app2 접속 성공



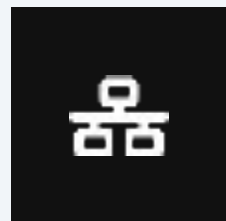
브라우저 접속 시 'Visit Site' 버튼을
클릭하여 ngrok 안내 화면을 통과하세요.

CONCLUSION

핵심 요약

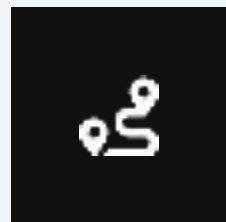
MetalLB

외부 IP 을 통한 클러스터 진입로 확보



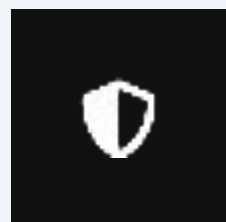
Ingress

URL 경로 분석을 통한 효율적인 수행



ngrok

인터넷 트래픽을 내부 서비스로 안전하게 터널링 연결



"하나의 도메인으로
복수의 서비스를
완벽하게 제어합니다."

K8s Ingress Controller & Connectivity Architecture

Practice #1

이전의 Docker 실습에서 ...

- docker login
- docker tag springboot-mysql-openjdk21-springboot-app:latest
wyhwang/spboot_app_jdk21:latest
- docker push wyhwang/spboot_app_jdk21:latest

실습

mysql-pvc가 mysql-pv 실습을 위해서 /mnt/data/mysql 경로는 테스트 환경에서 존재해야 하며, 필요 시 수동 생성:

```
sudo mkdir -p /mnt/data/mysql sudo chmod 777 /mnt/data/mysql
```

springboot-deployment.yaml에서 docker image 경로 지정 확인(docker hub id/image name:version)

```
containers:
  - name: springboot-app
    image: wyhwang/spboot_app_jdk21:latest # 빌드 후 Docker Hub에 올려야 함
    ports:
      - containerPort: 8080
```

실행 순서

```
kubectl apply -f mysql-volume.yaml
```

```
kubectl apply -f mysql-deployment.yaml
```

```
kubectl apply -f mysql-service.yaml
```

```
kubectl apply -f springboot-deployment.yaml
```

```
kubectl apply -f springboot-service.yaml
```


결과 확인

확인

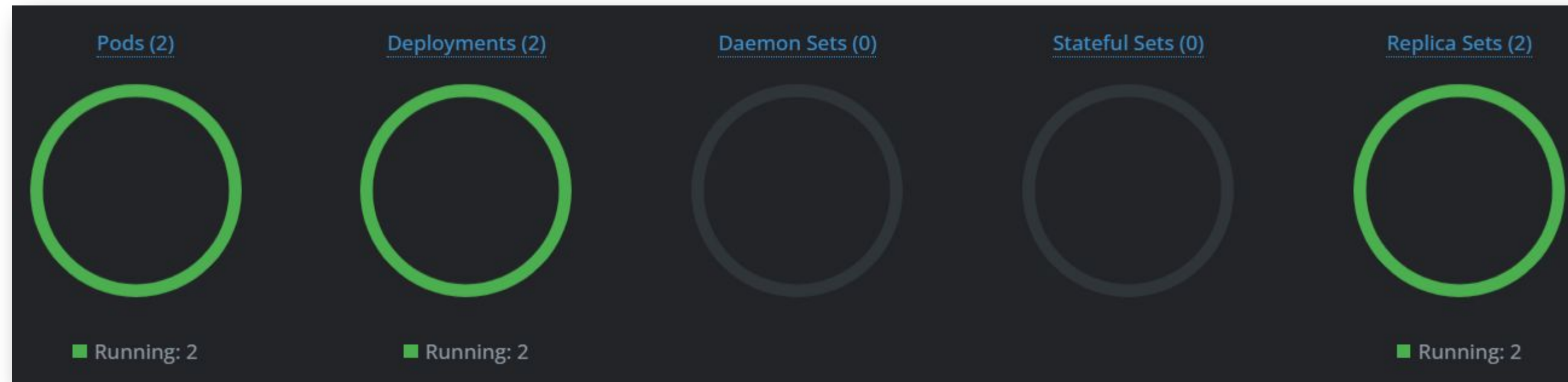
```
kubectl get svc springboot-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
springboot-app	NodePort	10.107.99.89	<none>	8087:30810/TCP	2m52s

```
curl http://192.168.56.61:30810
```

```
Hello from Spring Boot with OpenJDK 21 and MySQL!root@master:~/kopoDockerPractice/springboot-mysql-openjdk21/kubernetes
```

결과 확인



Storage

- Persistent Volume Claims
- Persistent Volumes**
- Storage Classes

☐	Name	Storage Class	Capacity	Claim	Age
☐	mysql-pv		1Gi	mysql-pvc	47m

Storage

- Persistent Volume Claims**
- Persistent Volumes
- Storage Classes

☐	Name	Namespace	Storage class	Size	Pods	Age
☐	mysql-pvc	default		1Gi	mysql-5c4f	46m

기타

[compose] Docker Compose를 Kubernetes YAML로 바꿔주는 자동화 도구

Kompose란?

- 공식 사이트: <https://kompose.io>
- GitHub: <https://github.com/kubernetes/kompose>

주요 목적:

Docker Compose로 작성된 프로젝트를 빠르게 Kubernetes 환경으로 마이그레이션하기 위함입니다.

Kompose 주요 기능

기능	설명
<code>docker-compose.yml</code> → <code>*.yaml</code>	Kubernetes Deployment, Service, PVC 등으로 자동 변환
네트워크, 환경변수, 볼륨 등 처리	Compose의 설정을 Kubernetes 형식에 맞게 변환
다양한 드라이버 지원	OpenShift, Helm 등으로도 변환 가능 (옵션)

사용 방법

- `docker-compose.yml` 이 있는 디렉토리로 이동
- 다음 명령어 실행:

```
bash
kompose convert
```

- 결과:

```
bash
mysql-service.yaml
mysql-deployment.yaml
springboot-app-deployment.yaml
springboot-app-service.yaml
```

- 적용:

```
bash
kubectl apply -f .
```

```
curl -L https://github.com/kubernetes/kompose/releases/download/v1.30.0/kompose-linux-amd64 -o kompose
```

```
chmod +x kompose
```

```
sudo mv kompose /usr/local/bin/
```


Tip & Trouble Shooting

Log search

- `kubectl logs [pod name] -n kube-system`
ex> `kubectl logs coredns-383838-fh38fh8 -n kube-system`
- `kubectl describe nodes`

When you change network plugin...

Pods failed to start after switch cni plugin from flannel to calico and then flannel

1 Answer

Active

Oldest

Votes

I use following steps to remove old calico configs from kubernetes without `kubeadm reset` :

1. clear ip route: `ip route flush proto bird`

2. remove all calico links in all nodes `ip link list | grep cali | awk '{print $2}' | cut -c 1-15 | xargs -I {} ip link delete {}`

3. remove ipip module `modprobe -r ipip`

4. remove calico configs `rm /etc/cni/net.d/10-calico.conflist && rm /etc/cni/net.d/calico-kubeconfig`

5. restart kubelet `service kubelet restart`

After those steps all the running pods won't be connect, then I have to delete all the pods, then all the pods works. This has litter influence if you are using `replicaset` .

share improve this answer follow

answered Dec 29 '18 at 0:03



aisensiy

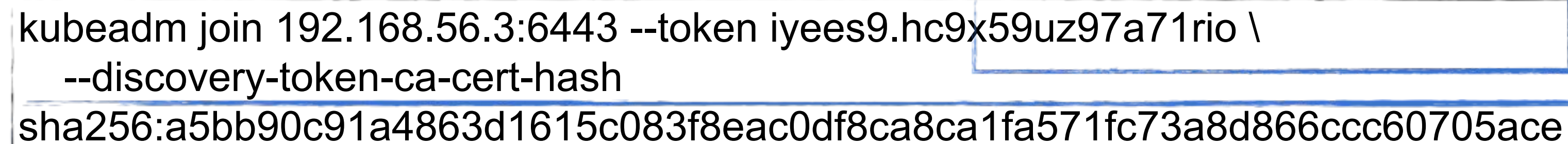
1,110 1 16 33

Reference :

<https://stackoverflow.com/questions/53900779/pods-failed-to-start-after-switch-cni-plugin-from-flannel-to-calico-and-then-fla>

When you lost token...

- kubeadm token list



```
kubeadm join 192.168.56.3:6443 --token iyees9.hc9x59uz97a71rio \  
--discovery-token-ca-cert-hash  
sha256:a5bb90c91a4863d1615c083f8eac0df8ca8ca1fa571fc73a8d866ccc60705ace
```

- openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt | openssl rsa -pubin -outform der 2>/dev/null | openssl dgst -sha256 -hex | sed 's/^.* //'
- If you want to create new token;
\$ kubeadm token create